

Programsko inženjerstvo ak.god 2025./2026.

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

QuizWiz

Tim: <TG09.1>

Ime tima: Wizionari

Nastavnik: Vlado Sruk

1.1 Uvod i cilj projekta

Projekt **QuizWiz** predstavlja web-aplikaciju namijenjenu digitalizaciji i modernizaciji organizacije pub kvizova, omogućujući jednostavnu komunikaciju i koordinaciju između organizatora i sudionika. Cilj projekta je razviti intuitivno, centralizirano rješenje koje omogućuje upravljanje kvizovima, prijave timova, evidenciju sudionika te obavještanje o promjenama u stvarnom vremenu.

Glavna svrha aplikacije je poboljšati transparentnost i učinkovitost organizacije pub kvizova, koji su danas često vođeni ručno ili putem društvenih mreža i chat grupa. Sustav **QuizWiz** omogućuje organizatorima jednostavno dodavanje događaja, odobravanje prijava i komunikaciju s timovima, dok korisnicima pruža mogućnost pretraživanja kvizova, prijava i informiranja o promjenama.

1.2 Opis problema i motivacija

U praksi, većina pub kvizova organizira se neformalno – putem objava na Facebook grupama ili privatnih poruka. Takav pristup dovodi do niza poteškoća:

- nedostatka centraliziranih informacija o dostupnim kvizovima,
- nemogućnosti praćenja broja prijavljenih timova i kapaciteta,
- nejasne komunikacije između organizatora i sudionika,
- otežanog informiranja o promjenama termina ili otkazivanju događaja.

Aplikacija **QuizWiz** adresira ove probleme pružajući objedinjenu platformu za sve dionike – administratore sustava, organizatore kvizova i igrače. Sustav uvodi automatizirane prijave, ograničavanje kapaciteta, notifikacije e-mailom i komentare ispod kvizova za brzu komunikaciju. Time se eliminira potreba za neformalnim kanalima i postiže bolja kontrola, preglednost i pouzdanost organizacije kvizova.

1.3 Potencijalna korist projekta

Projekt **QuizWiz** donosi višestruke koristi različitim skupinama korisnika:

- **Organizatorima** omogućuje jednostavno planiranje, objavu i administraciju događaja uz automatske obavijesti i praćenje prijava.
- **Sudionicima** nudi preglednu tražilicu kvizova, transparentne prijave, brzu komunikaciju s organizatorima i pravovremene obavijesti o promjenama.

- **Administratorima** sustava osigurava nadzor nad pravilnim radom sustava, dodavanje novih organizatora i održavanje baze mjesta kvizova.
- Vlasnicima ugostiteljskih objekata indirektno koristi povećanjem posjećenosti i promocijom događaja.

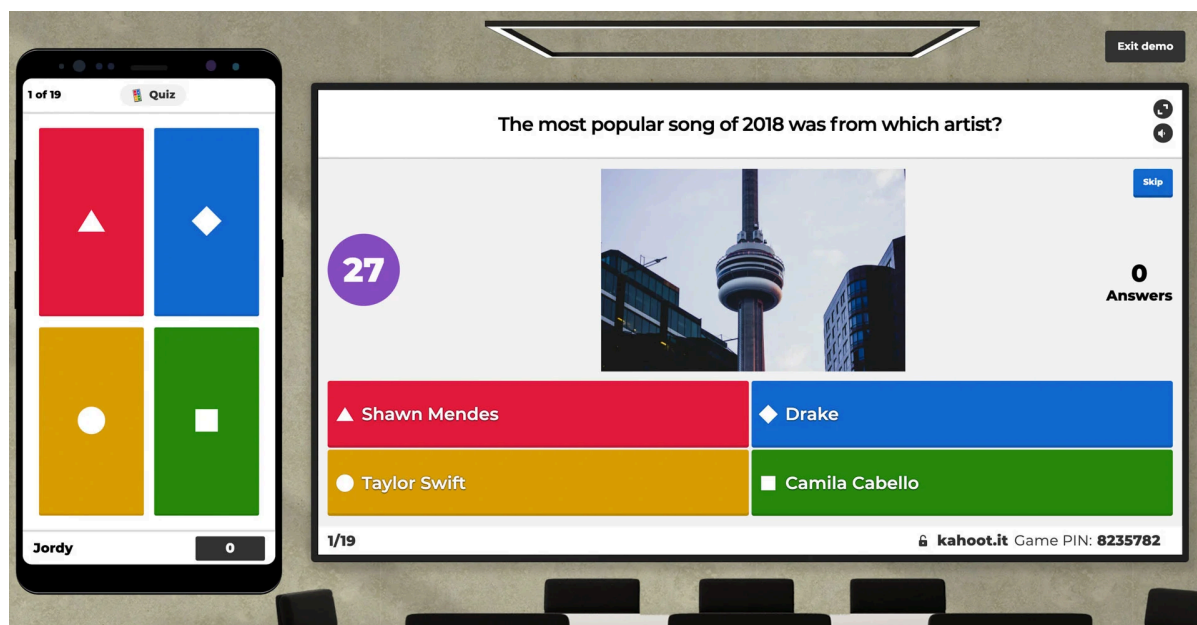
Dodatno, platforma može doprinijeti stvaranju zajednice kvizaša, potaknuti međusobno natjecanje i promovirati kulturu znanja i timskog duha.

1.4 Postojeća slična rješenja

Na tržištu postoji nekoliko platformi koje djelomično nude funkcionalnosti slične QuizWizu, no nijedna nije posebno prilagođena lokalnoj organizaciji pub kvizova uživo.

Kahoot!

Kahoot je globalno popularna aplikacija za interaktivne kvizove, najčešće korištena u obrazovanju. Njezina osnovna svrha je vođenje kvizova u stvarnom vremenu gdje korisnici odgovaraju putem mobilnih uređaja. Međutim, Kahoot nije namijenjen za prijave timova, upravljanje događajima ni komunikaciju između organizatora i sudionika.



kahoot

QuizPlanet

QuizPlanet je popularna mobilna aplikacija za online natjecanja u kvizovima između pojedinaca, no fokus joj je na natjecanju jedan-na-jedan, bez podrške za stvarne događaje ili timove. Projekt QuizWiz razlikuje se po tome što omogućuje fizičko sudjelovanje u lokalnim pub kvizovima i fokusira se na organizacijsku komponentu.

MyQuiz

MyQuiz je platforma za internetske kvizove i evente, često korištena za online timska natjecanja. Iako nudi upravljanje kvizovima i prijavama, fokusirana je na online okruženje, dok QuizWiz kombinira digitalnu prijavu i fizičko održavanje događaja u ugostiteljskim prostorima.

1.5 Skup korisnika

Sustav QuizWiz primarno je namijenjen trima skupinama - organizatorima kvizova i korisnicima (igračima) te administratorima sustava. Sekundarni korisnici mogu biti vlasnici ugostiteljskih objekata koji organiziraju kvizove te posjetitelji koji traže zanimljive događaje.

1.5.1 Uloge korisnika

1. Administrator sustava Administrator sustava zadužen je za održavanje ispravnosti rada aplikacije. On kreira i upravlja korisničkim računima organizatora, odobrava nove organizatore te po potrebi ažurira ili uklanja podatke o mjestima održavanja kvizova. Također ima uvid u sve kreirane događaje i korisničke aktivnosti radi osiguravanja pravilnog rada sustava.
2. Organizator kviza
 - kreira novi kviz s podacima: naslov, opis, datum, vrijeme, lokacija (Google Maps integracija), maksimalan broj timova, vrsta prijava (otvorene / uz odobrenje), oznake (npr. “opće znanje”, “glazbeni”, “tematski”),
 - odobrava ili odbija prijave za kvizove koji zahtijevaju verifikaciju,
 - otkazuje ili odgađa kviz uz automatsku e-mail obavijest svim prijavljenima,
 - prati broj prijavljenih timova i automatski prelazak kviza u stanje Popunjen kad se dosegne kapacitet.
3. Korisnik (igrač / tim)
 - pregledava i pretražuje kvizove po oznakama i datumu,
 - prijavljuje svoj tim na kviz (odmah ili uz odobrenje organizatora, ovisno o postavci),
 - prima obavijesti o odobrenju, otkazivanju ili odgodi kviza,
 - otkazuje prijavu tima prije održavanja kviza.

1.6 Opseg i mogućnost prilagodbe rješenja

Projekt u prvoj fazi uključuje osnovne funkcionalnosti: upravljanje kvizovima, prijave, komunikaciju i obavijesti. Sustav je modularan i može se proširivati bez izmjena postojeće arhitekture.

Mogućnosti prilagodbe uključuju: * dodavanje višejezične podrške, * prilagodbu dizajna različitim vrstama kvizova (glazbeni, tematski, opći), * mogućnost integracije s društvenim mrežama radi promocije događaja, * prilagodbu za mobilne uređaje s responzivnim dizajnom.

Sustav je zamišljen kao web-aplikacija s mogućnošću kasnijeg razvoja mobilne verzije.

1.7 Moguće nadogradnje

Potencijalne nadogradnje projekta uključuju: * dodavanje sustava ocjenjivanja i povratnih informacija nakon održanih kvizova, * sustav bodovanja i rang lista za timove i pojedince, * automatizirane kvizaške lige koje povezuju više događaja istog tipa, * analitički modul za organizatore (statistika prijave, posjećenosti i popularnosti), * mogućnost integracije s platnim sustavima za plaćene kvizove ili posebne evente.

2.1 Funkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
F-001	Sustav omogućuje registraciju	Visok	Zahtjev dionika	Ispunjavanje osnovnih informacija za kreiranje računa.
F-001.1	Registracija korisnika	Visok	Zahtjev dionika	Korisnici se mogu registrirati putem obrasca s osnovnim podacima (korisničko ime, lozinka, e-mail, naziv tima, kontakt podaci) i dobiti potvrdu o uspješnoj registraciji. Korisnik također može odabrati opciju registracije tima, dodaje naziv i članove tima.
F-001.2	Promocija usera u organizatore	Visok	Zahtjev dionika	Organizatori mogu kreirati profile.
F-002	Sustav omogućuje prijavu	Visok	Zahtjev dionika	Ispunjavanje potrebnih informacija za log-in u sustav.
F-002.1	Prijava korisnika	Visok	Zahtjev dionika	Korisnici se mogu prijaviti pomoću korisničkog imena ili e-maila i lozinke.
F-002.2	Prijava organizatora kvizova	Visok	Zahtjev dionika	Prijava organizatora kviza pomoću e-mail adrese i lozinke.
F-003	Uređivanje profila	Srednji	Zahtjev dionika	Omogućuje se uređivanje profila korisnika i/ili organizatora.
F-003.1	Uređivanje profila korisnika	Srednji	Zahtjev dionika	Korisnici mogu ažurirati korisničko ime i kontakt podatke.
F-003.2	Uređivanje profila tima	Srednji	Zahtjev dionika	Timovi mogu ažurirati naziv, članove i kontakt informacije.
F-003.3	Uređivanje profila organizatora	Srednji	Zahtjev dionika	Organizatori mogu ažurirati podatke (naziv, kontakt).
F-004	Kreiranje, uređivanje i brisanje postojećih kvizova	Visok	Zahtjev dionika	Sustav omogućuje kreiranje, uređivanje i brisanje postojećih kvizova te upravljanje prijavom timova na kvizove.
F-004.1	Kreiranje kvizova	Visok	Zahtjev dionika	Organizatori mogu uspješno kreirati i objavljivati kvizove s detaljima (naziv,

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
				opis, datum i vrijeme održavanja, lokacija, maksimalan broj timova, status prijava).
F-004.2	Uređivanje i brisanje postojećih kvizova	Visok	Zahtjev dionika	Organizatori mogu promijeniti ili obrisati kviz.
F-004.3	Upravljanje prijavama timova i obavijestima	Visok	Zahtjev dionika	Organizatori mogu ograničiti broj prijava, zatvoriti prijave i slati obavijesti o promjenama termina ili lokacije.
F-005	Pretraživanje kvizova prema datumu, lokaciji ili nazivu	Visok	Povratne informacije korisnika	Korisnici mogu filtrirati i pronaći kviz po datumu, lokaciji ili nazivu.
F-006	Prijava timova na kviz	Visok	Zahtjev dionika	Timovi se mogu prijaviti na kviz sve dok broj prijava ne premaši limit.
F-006.1	Pregled i praćenje prijava na kvizove	Visok	Povratne informacije korisnika	Timovi mogu vidjeti status prijava i primati obavijesti o promjenama.
F-006.2	Povlačenje prijave	Srednji	Zahtjev dionika	Sudionici mogu povući prijavu prije određenog roka.
F-007	Unos rezultata i rang lista	Srednji	Dokument zahtjeva	Organizatori mogu unijeti rezultate i rang liste (informacije su dostupne timovima) nakon završetka kviza.
F-007.1	Pregled rezultata i rang liste	Srednji	Povratne informacije korisnika	Korisnici imaju pristup informacijama o rezultatu i rang listi kviza nakon završetka.
F-008	Ocjenjivanje kvizova i organizatora	Srednji	Povratne informacije korisnika	Sudionici mogu ostaviti ocjenu i recenziju za kvizove i organizatore (prosječna ocjena se prikazuje za poboljšanje kvalitete).
F-008.1	Odgovor na recenzije i davanje povratnih informacija	Nizak	Zahtjev dionika	Organizatori mogu odgovarati na recenzije i davati povratne informacije.
F-009	Pregled povijesti sudjelovanja, rezultata i poretka	Srednji	Povratne informacije korisnika	Sudionici mogu pregledavati svoje prošle kvizove i rezultate (sustav prikazuje osobnu statistiku).

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
F-009.1	Prikaz arhive održanih kvizova i rezultata	Nizak	Dokument zahtjeva	Organizatori i sudionici mogu pristupiti arhivi i pregledavati rezultate kvizova.

2.2 Ostali (nefunkcionalni) zahtjevi

2.2.1 Zahtjevi održivosti

ID zahtjeva	Opis	Prioritet
NO-1	Sustav treba biti popraćen “Priručnikom za rad” koji opisuje pravilnu upotrebu aplikacije.	Visok
NO-2	Sustav treba imati dovoljnu dokumentaciju koda i arhitekture.	Srednji
NO-3	Aplikacija treba imati ugrađen alat za generiranje izvještaja o korištenju i prijavama.	Srednji
NO-4	Aplikacija treba biti jednostavna i intuitivna za korištenje.	Visok
NO-5	Sustav treba omogućiti jednostavno izvođenje nadogradnji bez prekida rada aplikacije.	Visok
NO-6	Aplikacija treba biti modularna, kako bi se lakše nadograđivala i prilagođavala novim zahtjevima ili integracijama s vanjskim sustavima.	Srednji

2.2.2 Zahtjevi skalabilnosti

ID zahtjeva	Opis	Prioritet
NS-1	Sustav treba podržavati minimalno 500 istovremenih korisnika (posebno tijekom objave popularnih kvizova) bez smanjenja performansi.	Visok
NS-2	Omogućiti asinkronu obradu složenijih operacija u pozadini (slanje obavijesti, generiranje statistike), čime se smanjuje vrijeme čekanja korisnika.	Visok
NS-3	Baza podataka i sam sustav treba podržavati horizontalnu skalabilnost, kako bi se smanjilo opterećenje na jednom serveru i omogućio paralelni rad više instanci baze i servera.	Visok

ID zahtjeva	Opis	Prioritet
NS-4	Sustav treba omogućiti lako dodavanje novih servera ili instanci baze podataka radi rasta korisnika.	Visok

2.2.3 Zahtjevi dostupnosti

ID zahtjeva	Opis	Prioritet
NPP-1	Sustav treba biti oblikovan tako da podržava jednostavno održavanje.	Visok
NPP-2	Aplikacija mora biti responzivna i usklađena s WCAG standardima pristupačnosti, kako bi bila dostupna svim korisnicima.	Visok
NPP-3	Implementirati strategije za visoku dostupnost: redundancija podataka i failover mehanizmi koji omogućuju nastavak rada u slučaju kvara.	Visok
NPP-4	Aplikacija treba imati ugrađeni centar za podršku i mogućnost slanja povratnih informacija putem e-maila.	Srednji

2.2.4 Sigurnosni zahtjevi

ID zahtjeva	Opis	Prioritet
NSG-1	Sustav mora implementirati višefaktorsku autentifikaciju za organizatore kvizova.	Visok
NSG-2	Osjetljivi podaci korisnika (lozinka, e-mail adrese) moraju biti šifrirani pri pohrani i prijenosu.	Visok
NSG-3	Sustav mora imati zaštitu od napada i drugih uobičajenih sigurnosnih prijetnji.	Srednji
NSG-4	Sustav mora omogućiti upravljanje pravima pristupa i razlikovanje uloga korisnika.	Srednji

2.2.5 Zahtjevi performansi

ID zahtjeva	Opis	Prioritet
NP-1	Sustav treba omogućiti brzo učitavanje stranica kviza za sve korisnike.	Visok
NP-2	Aplikacija treba prikazati rezultate korisničkih akcija (prijava na kviz, pregled rezultata,...) u roku 1-2 sekunde.	Visok
NP-3	Aplikacija treba biti optimizirana kako bi učinkovito koristila resurse poslužitelja i smanjila opterećenje procesora i memorije, osiguravajući stabilan rad čak i uz veliko	Srednji

ID zahtjeva	Opis	Prioritet
	opterećenje.	
NP-4	Sustav treba automatski optimizirati slike, lokacije i medijske sadržaje kviza radi smanjenja vremena učitavanja.	Srednji
NP-5	Sustav treba podržavati generiranje izvještaja i statistike bez primjetnog usporavanja.	Srednji

2.2.6 Zahtjevi ograničenja

ID zahtjeva	Opis	Prioritet
NOG-1	Omogućiti prijavu samo timovima iz regije ili države u kojoj je aplikacija.	Srednji
NOG-2	Postaviti ograničenje broja prijava koje jedan tim može podnijeti u kratkom vremenskom razdoblju (npr. jednu prijavu svakih 30 minuta).	Nizak

2.2.7 Zahtjevi interoperabilnosti

ID zahtjeva	Opis	Prioritet
NKU-1	Sustav treba biti kompatibilan sa svim modernim preglednicima.	Visok
NKU-2	Sustav treba nuditi obavijesti putem e-maila i push notifikacija.	Srednji
NKU-3	Sustav treba omogućiti jednostavno pretraživanje kvizova po oznakama, lokaciji i datumu.	Srednji

2.3 Dionici

1. sudionici/timovi (igrači) - korisnici koji sudjeluju u kvizovima, pregledavaju rezultate i primaju obavijesti
2. organizatori kvizova - korisnici koji kreiraju i uređuju kvizove, upravljaju prijavama i rezultatima te šalju obavijesti
3. administrator sustava - zadužen za održavanje sustava, upravljanje korisnicima i nadzor događaja
4. naručitelj - Vlado Sruk
5. razvojni tim - Wizionari

2.4 Aktori

- A1: sudionici/timovi
 - registracija ili kreiranje tima - F-001.1
 - prijava u sustav - F-002.1
 - pretraživanje kvizova po datumu, lokaciji ili oznakama - F-005, NKU-3
 - prijavljivanje tima na kviz (odmah ili uz odobrenje organizatora) - F-006
 - pregled i praćenje statusa prijave - F-006.1
 - povlačenje prijave prije roka - F-006.2
 - ocjenjivanje kvizova i organizatora - F-008
 - primanje obavijesti o statusu prijave, odgodi ili otkazu kviza - NKU-2
 - pregled arhive održanih kvizova i osobne statistike - F-009, F-009.1
- A2: organizator kviza
 - registracija kao organizator - F-001.2
 - prijava u sustav kao organizator - F-002.2
 - uređivanje profila organizatora - F-003.3
 - kreiranje, uređivanje i brisanje kvizova - F-004, F-004.1, F-004.2
 - upravljanje prijavama tima i slanje obavijesti - F-004.3
 - unošenje rezultata i rang liste - F-007
 - odgovaranje na recenzije i davanje povratnih informacija - F-008.1
 - pristup arhivi održanih kvizova i rezultata - F-009.1
- A3: administrator sustava
 - upravljanje korisničkim računima organizatora i odobravanje novih organizatora - F-001.2, NSG-4
 - ažuriranje ili uklanjanje podataka o mjestima održavanja kvizova - F-004, NO-5
 - praćenje svih kreiranih događaja i aktivnosti korisnika - NS-2
 - održavanje sigurnosti, dostupnosti i kompatibilnosti sustava - NPP-3, NSG-1, NSG-2
- A4: razvojni tim/naručitelj
 - nadziranje razvoja novih funkcionalnosti i modularnosti sustava - NO-6
 - optimiziranje performansi i responzivnosti - NP-1, NP-2, NP-3, NP-4
 - implementacija skalabilnosti i asinkrone obrade zahtjeva - NS-3, NS-4
 - planiranje nove verzije sustava i integracije - F-004.1, NKU-1, NKU-2

3.1 Obrasci uporabe

UC1 - Registracija korisnika/tima

- Glavni sudionik: Sudionik/tim
- Cilj: Registracija novog korisnika ili tima u sustav
- Sudionici: Baza podataka
- Preduvjet: —
- Referenca: *F-001, F-001.1, F-001.2*

Opis osnovnog tijeka:

1. Sudionik/tim otvara obrazac za registraciju.
2. Unosi potrebne podatke (korisničko ime, e-mail, lozinka, naziv tima, članovi, kontakt podaci).
3. Odabire opciju “Registriraj me”.
4. Sustav provjerava ispravnost i jedinstvenost podataka.
5. Sustav sprema podatke u bazu i šalje obavijest o uspješnoj registraciji putem e-maila.

Opis mogućih odstupanja:

- a. Korisnik je ispunio neki podatak nepravilno.
 - i. Sustav javlja grešku i označava nepravilno polje.
- b. E-mail ili korisničko ime već postoji.
 - i. Sustav javlja korisniku da su podaci zauzeti i nudi prijavu.

UC2 - Prijava u sustav

- Glavni sudionik: Sudionik/tim
- Cilj: Prijava korisnika u sustav
- Sudionici: Baza podataka
- Preduvjet: Korisnik mora biti registriran
- Referenca: *F-002, F-002.1, F-002.2*

Opis osnovnog tijeka:

1. Sudionik/tim otvara obrazac za prijavu.
2. Unosi korisničko ime ili e-mail i lozinku.
3. Odabire opciju “Prijavi me”.
4. Sustav provjerava ispravnost unesenih podataka.
5. Sustav učitava početnu stranicu sustava s prikazom korisničkog profila.

Opis mogućih odstupanja:

- a. Podaci su pogrešno uneseni.
- i. Sustav javlja pogrešku i označava problematično polje.
- b. Račun nije aktiviran.
- i. Sustav obavještava korisnika da mora potvrditi e-mail.

UC3 - Odjava iz sustava

- Glavni sudionik: Sudionik/tim
- Cilj: Odjava prijavljenog korisnika iz sustava
- Sudionici: —
- Preduvjet: Korisnik mora biti prijavljen
- Referenca: —

Opis osnovnog tijeka:

1. Sudionik/tim odabire opciju “Odjava”.
2. Sustav prekida korisničku sesiju.
3. Korisniku se učitava početna stranica aplikacije.
4. Sustav prikazuje poruku “Odjava uspješna”.

Opis mogućih odstupanja:

- a. Tijekom odjave nastane tehnička pogreška.
- i. Sustav obavještava korisnika da odjava nije dovršena.

UC4 - Uređivanje profila

- Glavni sudionik: Sudionik/tim, Organizator kviza, Administrator sustava
- Cilj: Ažurirati korisničke podatke profila
- Sudionici: Baza podataka
- Preduvjet: Korisnik mora biti prijavljen
- Referenca: *F-003, F-003.1, F-003.2, F-003.3*

Opis osnovnog tijeka:

1. Korisnik otvara svoj profil.
2. Odabire opciju “Uredi profil”.
3. Ažurira željene podatke (npr. korisničko ime, kontakt informacije, opis, lokaciju).
4. Odabire “Spremi promjene”.
5. Sustav sprema promjene u bazu i prikazuje poruku o uspješnom ažuriranju.

Opis mogućih odstupanja:

- a. Uneseni podaci nisu valjani.
 - i. Sustav označava pogrešna polja i traži ispravak.
- b. Došlo je do greške pri spremanju.
 - i. Sustav prikazuje poruku o neuspjelom ažuriranju profila.

UC5 - Brisanje korisničkog računa

- Glavni sudionik: Sudionik/tim
- Cilj: Brisanje vlastitog korisničkog računa iz sustava
- Sudionici: Baza podataka
- Preduvjet: Korisnik mora biti prijavljen
- Referenca: —

Opis osnovnog tijeka:

1. Sudionik/tim otvara postavke računa.
2. Odabire opciju “Obriši račun”.
3. Sustav traži potvrdu brisanja.
4. Nakon potvrde, sustav briše korisničke podatke iz baze i odjavljuje korisnika.
5. Korisnik prima potvrdu o brisanju računa putem e-maila.

Opis mogućih odstupanja:

- a. Korisnik odustane od brisanja.
 - i. Sustav prekida proces i vraća korisnika na profil.
- b. Sustav ne može obrisati račun zbog tehničke greške.
 - i. Sustav obavještava korisnika o neuspjelom brisanju.

UC6 - Pretraživanje kvizova

- Glavni sudionik: Sudionik/tim
- Cilj: Pretraživanje kvizova prema kriterijima
- Sudionici: Baza podataka
- Preduvjet: —
- Referenca: *F-005, NKU-3*

Opis osnovnog tijeka:

1. Sudionik/tim otvara stranicu s kvizovima.
2. Unosi kriterije pretrage (naziv, lokacija, datum).

3. Sustav prikazuje popis kvizova koji odgovaraju upitu.
4. Korisnik može otvoriti detalje pojedinog kviza.

Opis mogućih odstupanja:

- a. Nema kvizova koji odgovaraju kriterijima.
- i. Sustav prikazuje poruku “Nema pronađenih rezultata”.

UC7 - Prijava tima na kviz

- Glavni sudionik: Sudionik/tim
- Cilj: Prijava tima na kviz
- Sudionici: Baza podataka
- Preduvjet: Korisnik mora biti prijavljen
- Referenca: *F-006, F-006.1*

Opis osnovnog tijeka:

1. Sudionik/tim otvara stranicu kviza.
2. Odabire opciju “Prijavi tim”.
3. Sustav provjerava dostupnost mjesta i status prijava.
4. Sustav sprema prijavu i prikazuje poruku o uspjehu.

Opis mogućih odstupanja:

- a. Kviz je popunjen.
- i. Sustav onemogućuje prijavu i prikazuje poruku.
- b. Kviz je otkazan.
- i. Sustav prikazuje obavijest i ne dopušta prijavu.

UC8 - Povlačenje prijave

- Glavni sudionik: Sudionik/tim
- Cilj: Povlačenje prethodno poslane prijave na kviz
- Sudionici: Baza podataka
- Preduvjet: Korisnik mora imati aktivnu prijavu
- Referenca: *F-006.2*

Opis osnovnog tijeka:

1. Sudionik/tim otvara listu prijava.
2. Odabire kviz i opciju “Povuci prijavu”.
3. Sustav traži potvrdu radnje.

4. Sustav ažurira status prijave na “Povučen”.
5. Sustav prikazuje poruku o uspješnom povlačenju.

Opis mogućih odstupanja:

- a. Rok za povlačenje je istekao.
- i. Sustav onemogućuje povlačenje i prikazuje upozorenje.

UC9 - Pregled statusa prijave

- Glavni sudionik: Sudionik/tim
- Cilj: Pregled statusa prijavljenih kvizova
- Sudionici: Baza podataka
- Preduvjet: Korisnik mora biti prijavljen
- Referenca: *F-006.1*

Opis osnovnog tijeka:

1. Sudionik/tim otvara stranicu “Moje prijave”.
2. Sustav prikazuje popis svih prijavljenih kvizova.
3. Statusi prijava (potvrđeno, odbijeno, na čekanju) su vidljivi korisniku.

Opis mogućih odstupanja:

- a. Korisnik nema prijava.
- i. Sustav prikazuje poruku “Još niste prijavljeni ni na jedan kviz.”

UC10 - Pregled rezultata i arhive

- Glavni sudionik: Sudionik/tim
- Cilj: Pregled završenih kvizova i rezultata
- Sudionici: Baza podataka
- Preduvjet: Rezultati moraju biti uneseni
- Referenca: *F-007, F-009*

Opis osnovnog tijeka:

1. Sudionik/tim otvara sekciju “Rezultati”.
2. Odabire kviz iz arhive.
3. Sustav prikazuje rang listu i rezultate.

Opis mogućih odstupanja:

- a. Još nema rezultata za odabrani kviz.
- i. Sustav prikazuje poruku “Rezultati još nisu objavljeni.”

UC11 - Ocjenjivanje kvizova i organizatora

- Glavni sudionik: Sudionik/tim
- Cilj: Ostaviti ocjenu i recenziju kviza ili organizatora
- Sudionici: Baza podataka
- Preduvjet: Korisnik mora biti sudjelovao na kvizu
- Referenca: *F-008, F-008.1*

Opis osnovnog tijeka:

1. Sudionik/tim otvara kviz u kojem je sudjelovao.
2. Odabire opciju “Ocijeni kviz” ili “Ocijeni organizatora”.
3. Unosi ocjenu i opcionalni komentar.
4. Sustav sprema ocjenu i ažurira prosjek ocjena.

Opis mogućih odstupanja:

- a. Korisnik nije sudjelovao na kvizu.
 - i. Sustav ne dopušta unos ocjene.
- b. Recenzija sadrži neprimjeren sadržaj.
 - i. Sustav odbija recenziju i prikazuje poruku o kršenju pravila.

UC12 - Kreiranje kviza

- Glavni sudionik: Organizator kviza
- Cilj: Kreirati i objaviti novi kviz
- Sudionici: Baza podataka
- Preduvjet: Organizator mora biti prijavljen
- Referenca: *F-004, F-004.1*

Opis osnovnog tijeka:

1. Organizator otvara obrazac za novi kviz.
2. Unosi podatke o kvizu (naziv, opis, datum, vrijeme, lokaciju, broj timova).
3. Odabire opciju “Objavi”.
4. Sustav sprema podatke i prikazuje poruku o uspješnom kreiranju kviza.

Opis mogućih odstupanja:

- a. Nisu uneseni svi potrebni podaci.
 - i. Sustav označava obavezna polja i traži ispravak.

UC13 - Uređivanje/brisanje kviza

- Glavni sudionik: Organizator kviza
- Cilj: Uređivanje ili brisanje postojećeg kviza
- Sudionici: Baza podataka
- Preduvjet: Organizator mora biti vlasnik kviza
- Referenca: *F-004.2*

Opis osnovnog tijeka:

1. Organizator otvara stranicu svog kviza.
2. Odabire opciju “Uredi” ili “Obriši kviz”.
3. Sustav provjerava prava pristupa.
4. Sustav sprema izmjene ili briše kviz i prikazuje potvrdu.

Opis mogućih odstupanja:

- a. Kviz ima aktivne prijave.
- i. Sustav onemogućuje brisanje i nudi opciju “Odgodi kviz”.

UC14 - Upravljanje prijavama

- Glavni sudionik: Organizator kviza
- Cilj: Upravljanje prijavama timova na kviz
- Sudionici: Baza podataka
- Preduvjet: Organizator mora imati objavljen kviz
- Referenca: *F-004.3*

Opis osnovnog tijeka:

1. Organizator otvara listu prijava za svoj kviz.
2. Pregledava pristigle prijave i podatke o timovima.
3. Odobrava ili odbija prijave.
4. Sustav ažurira statute i šalje obavijesti prijavljenima.

Opis mogućih odstupanja:

- a. Broj prijava premašuje maksimalni limit.
- i. Sustav automatski zatvara prijave i prikazuje obavijest.

UC15 - Unos rezultata i rang liste

- Glavni sudionik: Organizator kviza
- Cilj: Unos rezultata kviza i prikaz rang liste
- Sudionici: Baza podataka

- Preduvjet: Kviz mora biti završen
- Referenca: *F-007*

Opis osnovnog tijeka:

1. Organizator otvara završeni kviz.
2. Unosi rezultate i bodove po timovima.
3. Sustav automatski generira rang listu.
4. Rezultati postaju vidljivi sudionicima.

Opis mogućih odstupanja:

- a. Nisu uneseni svi rezultati.
- i. Sustav upozorava korisnika i omogućava djelomično spremanje.

UC16 - Odobravanje organizatora i upravljanje korisnicima

- Glavni sudionik: Administrator sustava
- Cilj: Upravljanje korisnicima i odobriti nove organizatore
- Sudionici: Baza podataka
- Preduvjet: Administrator mora biti prijavljen
- Referenca: *NSG-4*

Opis osnovnog tijeka:

1. Administrator otvara konzolu za upravljanje korisnicima.
2. Pregledava zahtjeve za ulogu organizatora.
3. Odobrava ili odbija zahtjeve.
4. Može uređivati ili deaktivirati korisnike.
5. Sustav sprema promjene i šalje obavijesti korisnicima.

Opis mogućih odstupanja:

- a. Administrator nema potrebne ovlasti.
- i. Sustav blokira akciju i prikazuje upozorenje.

UC17 - Slanje obavijesti

- Glavni sudionik: Sustav (automatizirano)
- Cilj: Obavješćavanje korisnika o promjenama ili događajima
- Sudionici: Servis za e-mail i push notifikacije
- Preduvjet: Postoji događaj koji zahtijeva obavijest
- Referenca: *NKU-2*

Opis osnovnog tijeka:

1. Sustav prepoznaje događaj (promjena termina, nova prijava, rezultat).
2. Generira sadržaj obavijesti.
3. Šalje obavijest korisnicima putem e-maila ili push notifikacije.
4. Bilježi status isporuke.

Opis mogućih odstupanja:

- a. Servis za slanje obavijesti nije dostupan.
- i. Sustav sprema obavijest u red čekanja i pokušava ponovno kasnije.

UC18 - Brisanje recenzija

- Glavni sudionik: Administrator sustava
- Cilj: Ukloniti neprimjerene recenzije ili komentare
- Sudionici: Baza podataka
- Preduvjet: Administrator mora biti prijavljen
- Referenca: *F-008.1*

Opis osnovnog tijeka:

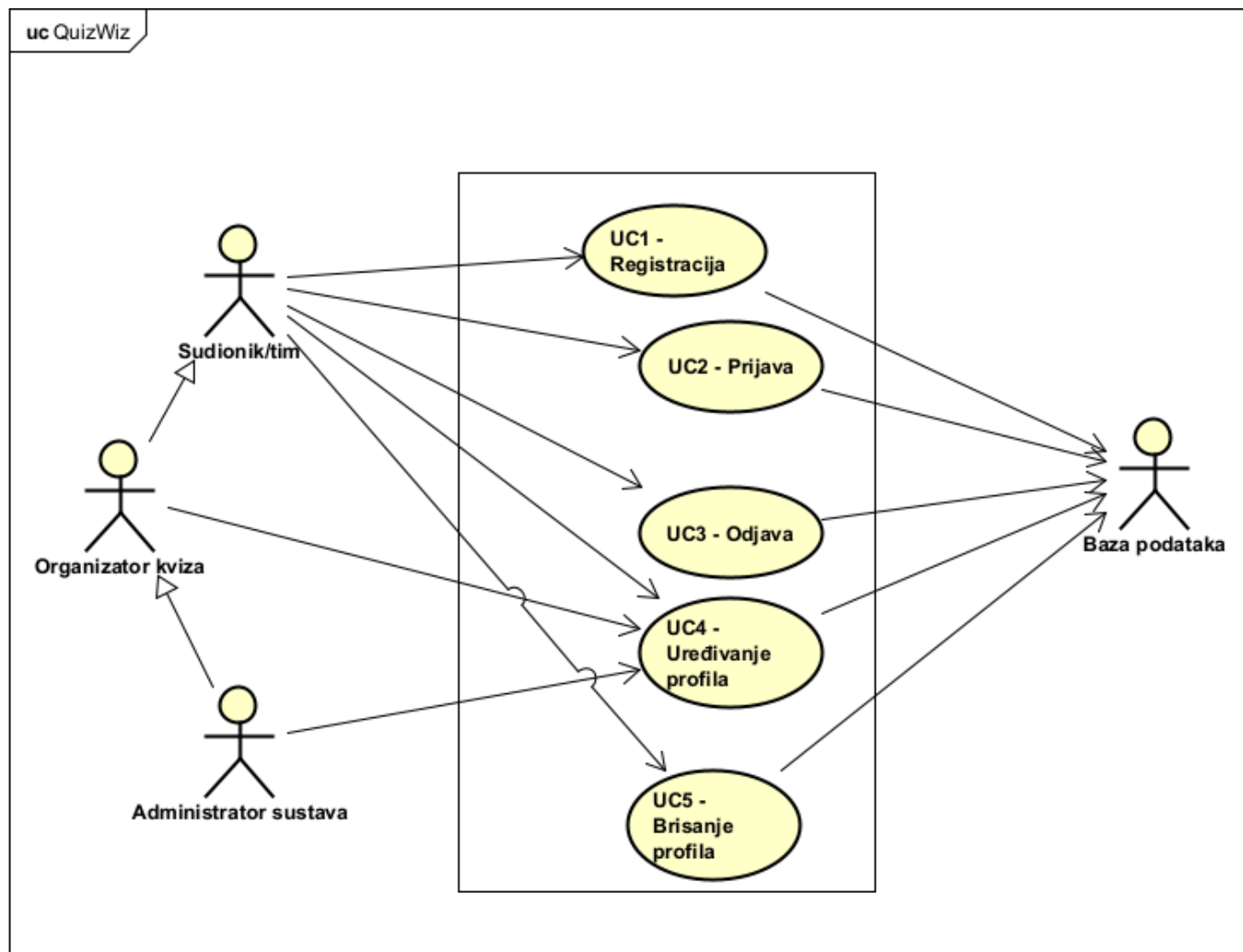
1. Administrator otvara sekciju recenzija.
2. Pregledava sve prijavljene ili označene recenzije.
3. Odabire opciju “Obriši recenziju”.
4. Sustav uklanja recenziju iz baze i obavještava korisnika o brisanju.

Opis mogućih odstupanja:

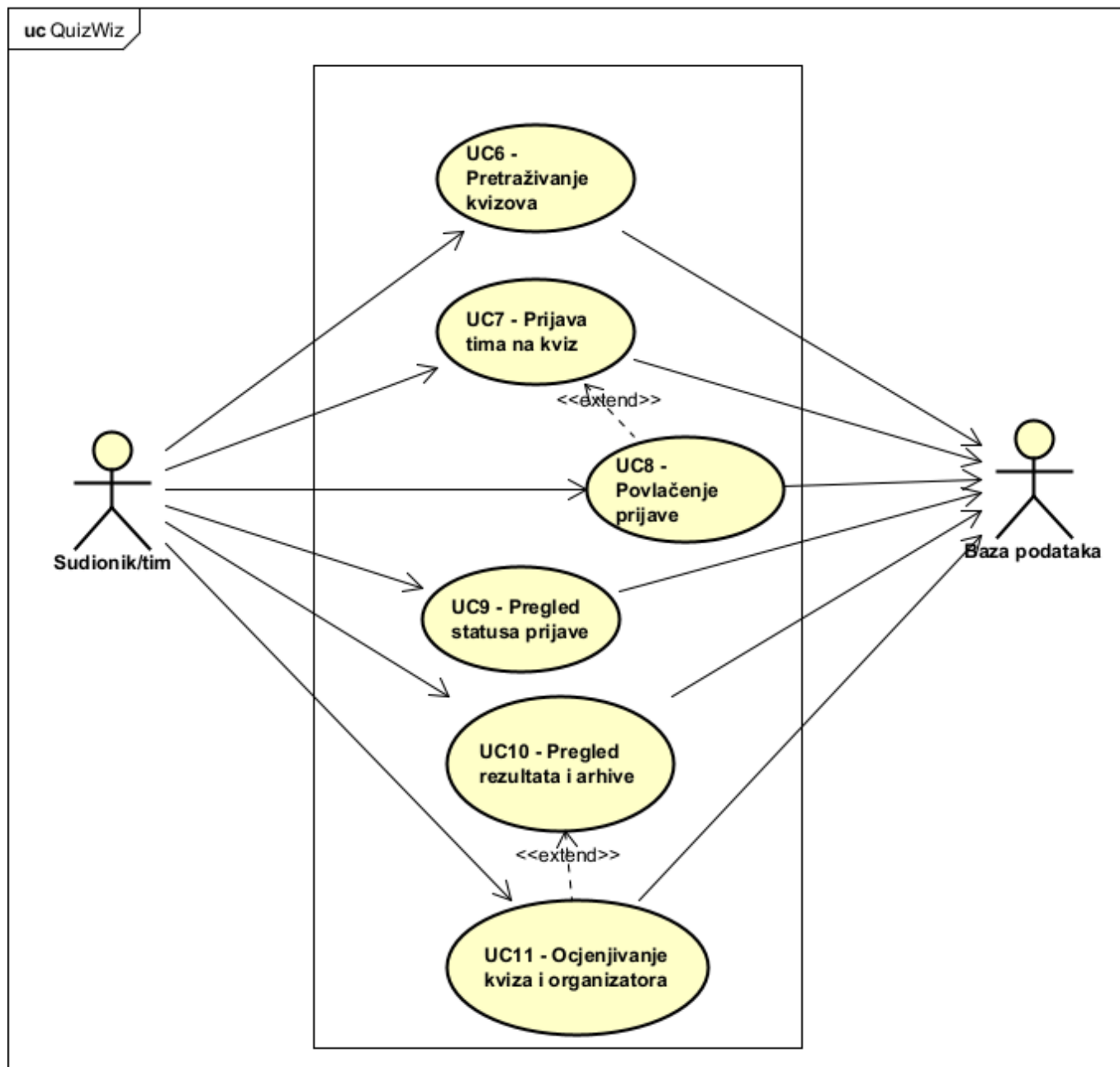
- a. Recenzija je već uklonjena.
- i. Sustav prikazuje poruku “Recenzija ne postoji.”
- b. Administrator nema dovoljne ovlasti.
- i. Sustav onemogućuje brisanje i prikazuje upozorenje.

3.2 Dijagrami obrazaca uporabe

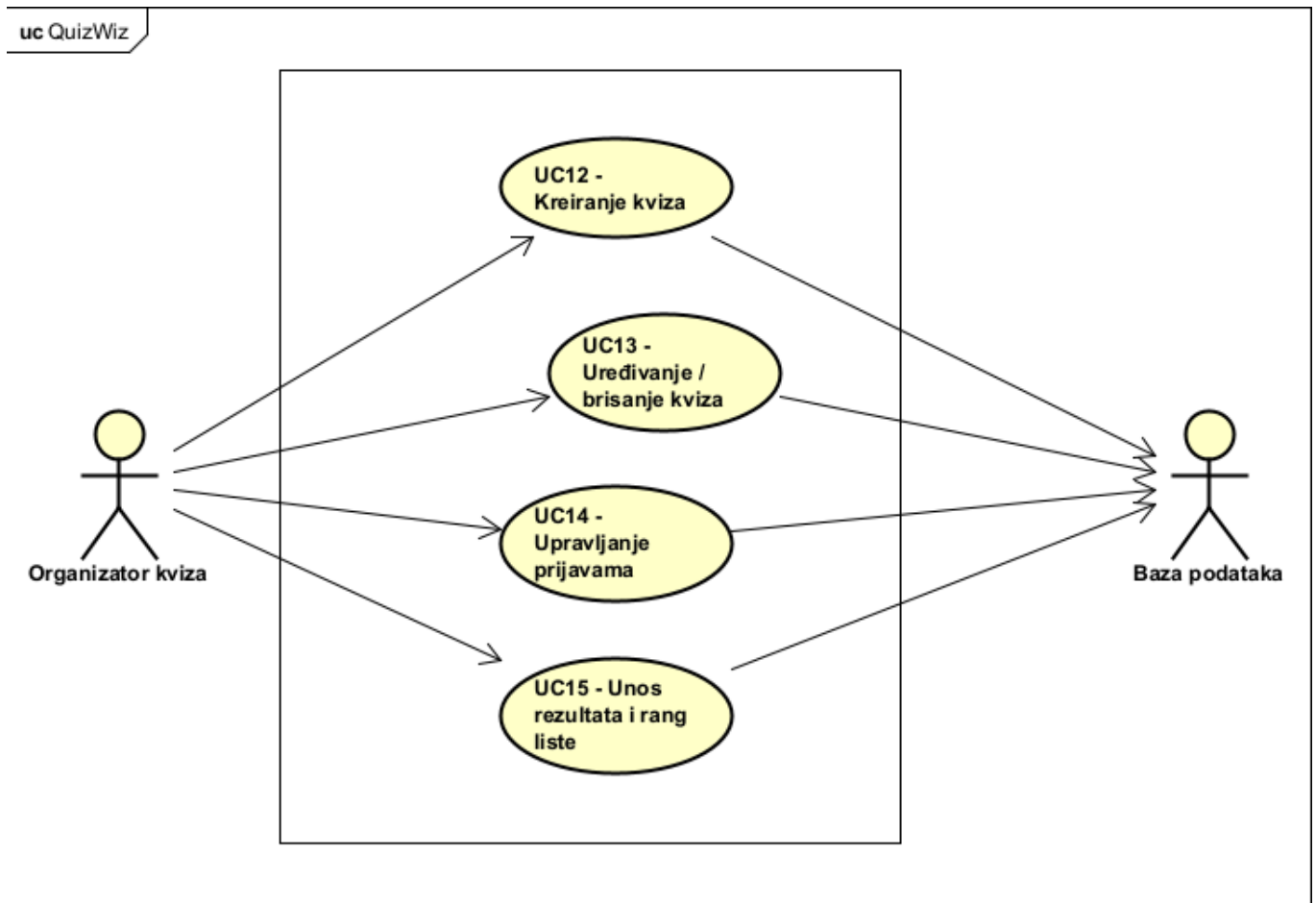
3.2.1 UC dijagram za korisničke račune



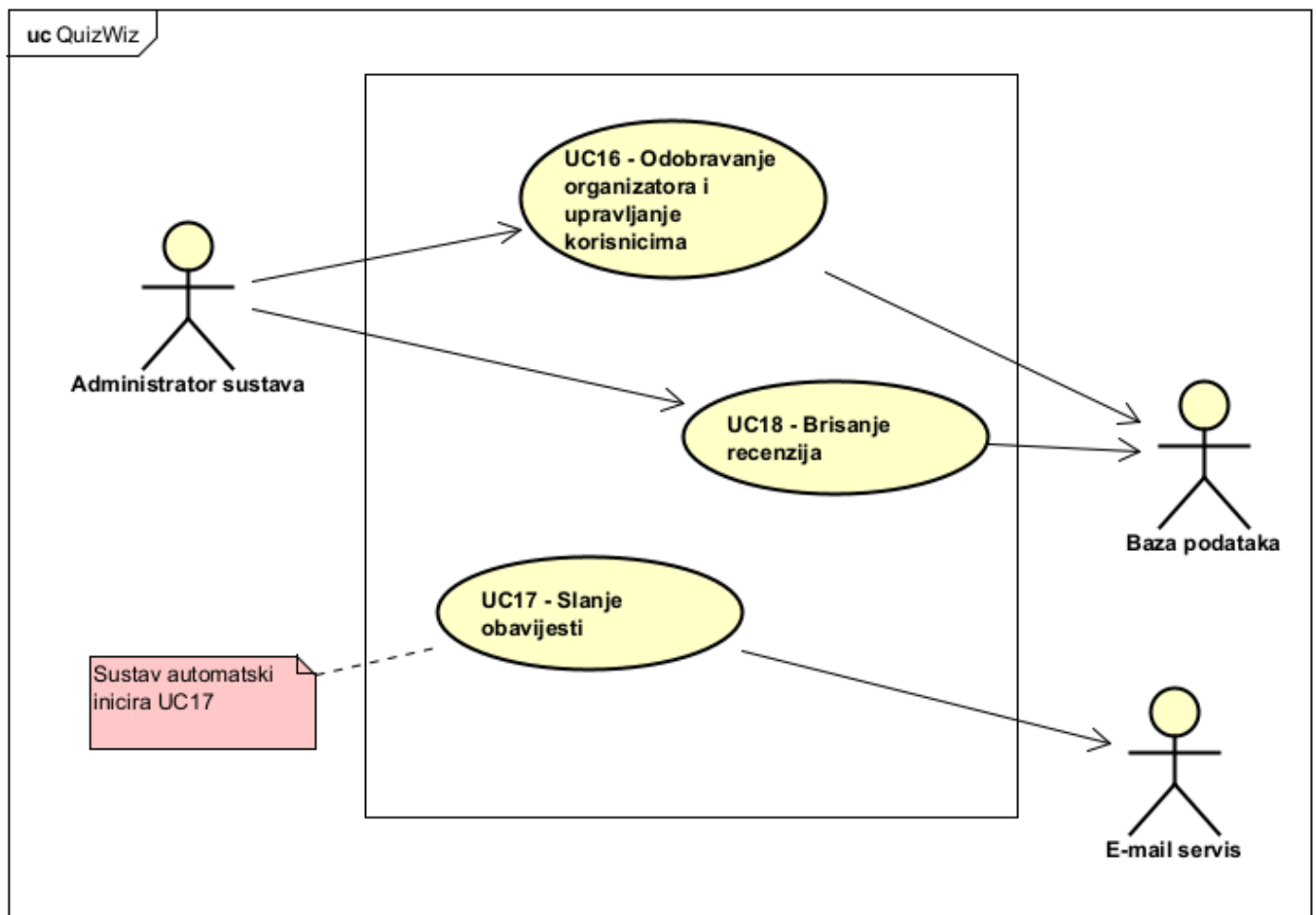
3.2.2 UC dijagram funkcionalnosti sudionika/tima



3.2.3 UC dijagram funkcionalnosti organizatora kviza



3.2.4 UC dijagram funkcionalnosti administratora i sustava

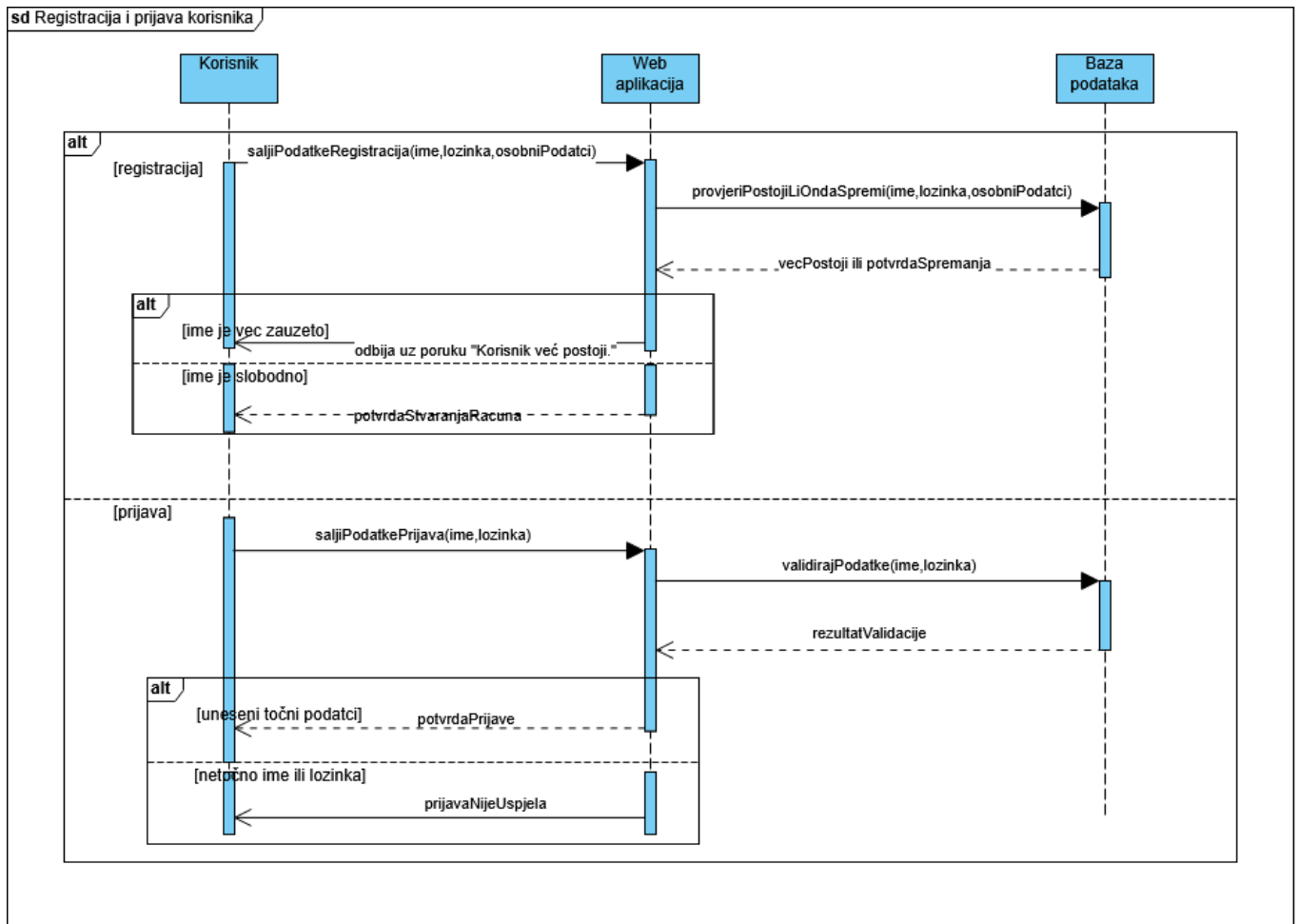


3.3 Sekvencijski dijagrami

UC1, UC2 Neprijavljeni korisnik - registracija i prijava korisnika u sustav

Ako je odabrana registracija, neprijavljeni korisnik tada unosi željeno ime, lozinku i odgovarajuće osobne podatke. Aplikacija proslijeđuje podatke prema bazi podataka, gdje se provjerava postoji li već korisnik s istim imenom. Ukoliko uneseno ime ne postoji, u bazu podataka spremaju se podatci o korisniku, a korisniku se javlja da je registracija uspješna. Ukoliko već postoji ime u bazi podataka, javlja se korisniku da registracija nije uspješna.

Ako je odabrana prijava, neprijavljeni korisnik unosi svoje ime i lozinku, koji se proslijeđuju i validiraju. U slučaju da lozinka odgovara korisničkom računu, javlja se korisniku da je prijava uspješna, a u suprotnom da je uneseno krivo ime ili lozinka te da je prijava neuspješna.



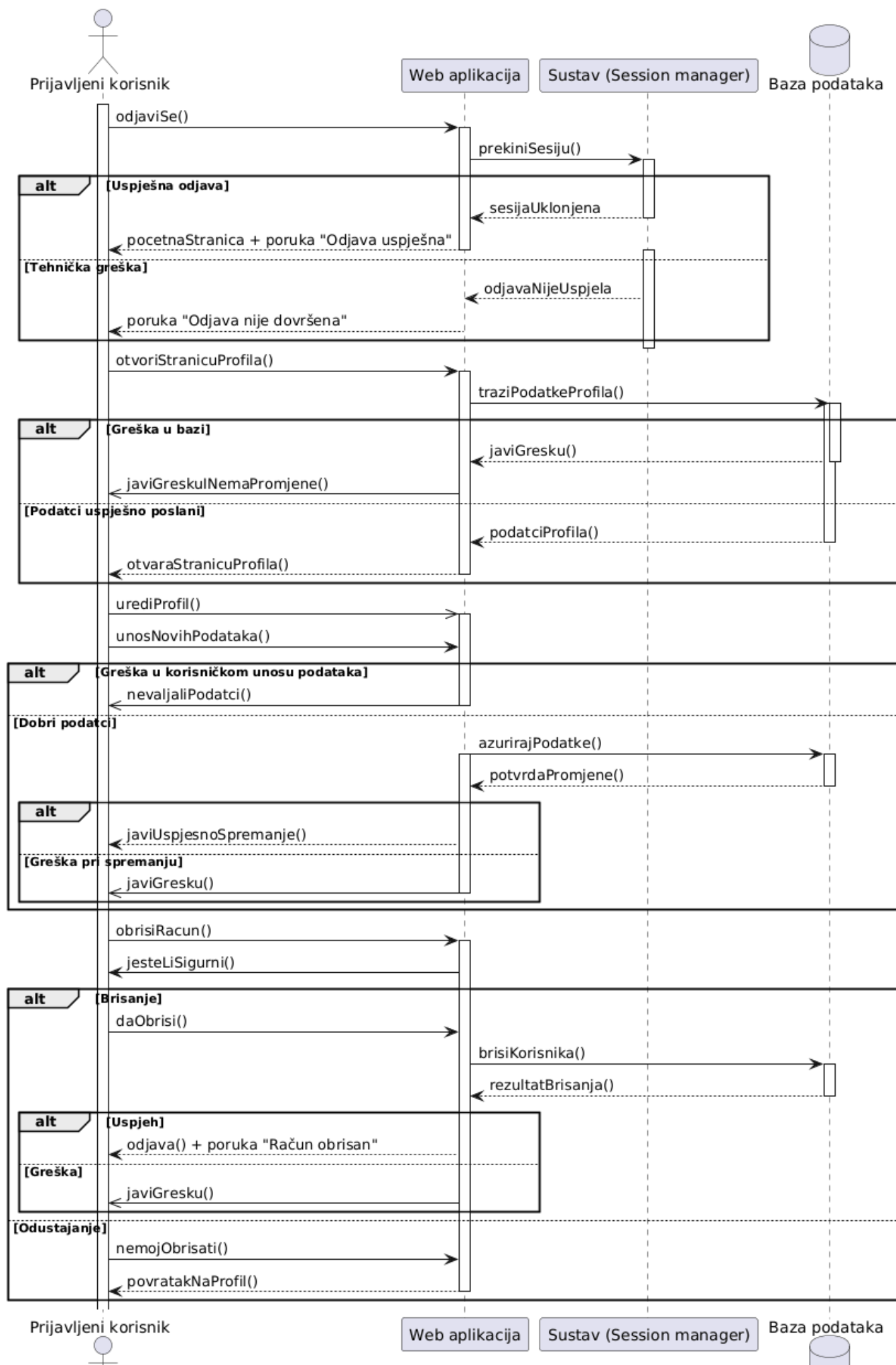
UC3, UC4, UC4 Prijavljeni korisnik - odjava, uređivanje i brisanje računa

Dijagram prikazuje osnovne radnje prijavljenog korisnika: odjavu, uređivanje i brisanje računa.

Korisnik se može odjaviti iz sustava, pri čemu aplikacija prekida sesiju i vraća poruku o uspjehu ili grešci. Na profilnoj stranici aplikacija dohvaća podatke iz baze; ako je došlo do pogreške, prikazuje se poruka o neuspjehu, a ako je uspješno, otvara se profil.

Pri uređivanju profila korisnik unosi nove podatke. Ako su podaci neispravni, sustav javlja pogrešku, a ako su ispravni, ažurira ih u bazi i prikazuje poruku o uspješnom spremanju.

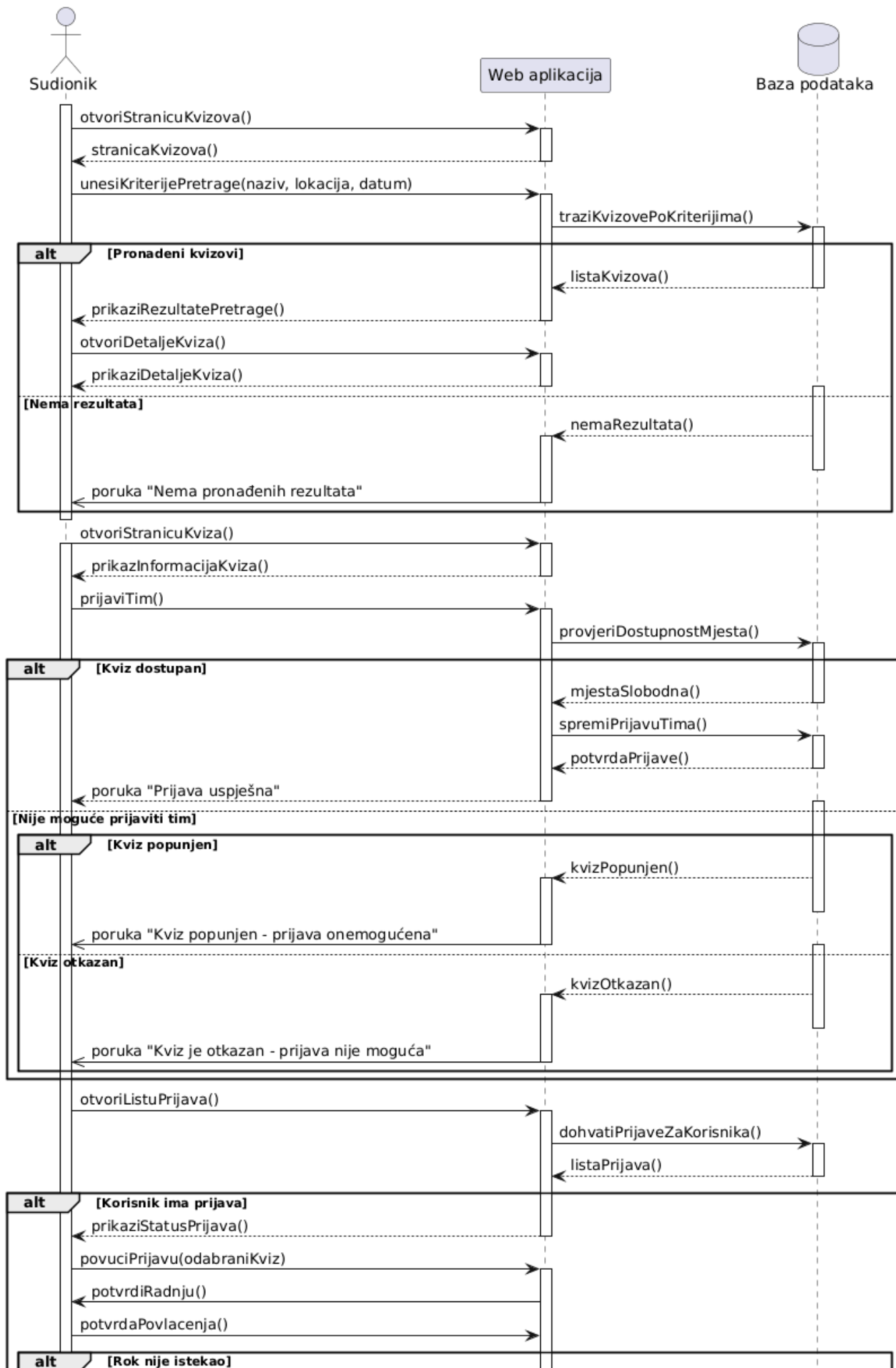
Korisnik može i obrisati račun. Sustav traži potvrdu, a ako korisnik pristane, briše račun iz baze i odjavljuje ga uz poruku o uspjehu; u suprotnom, vraća ga na profil bez promjena.

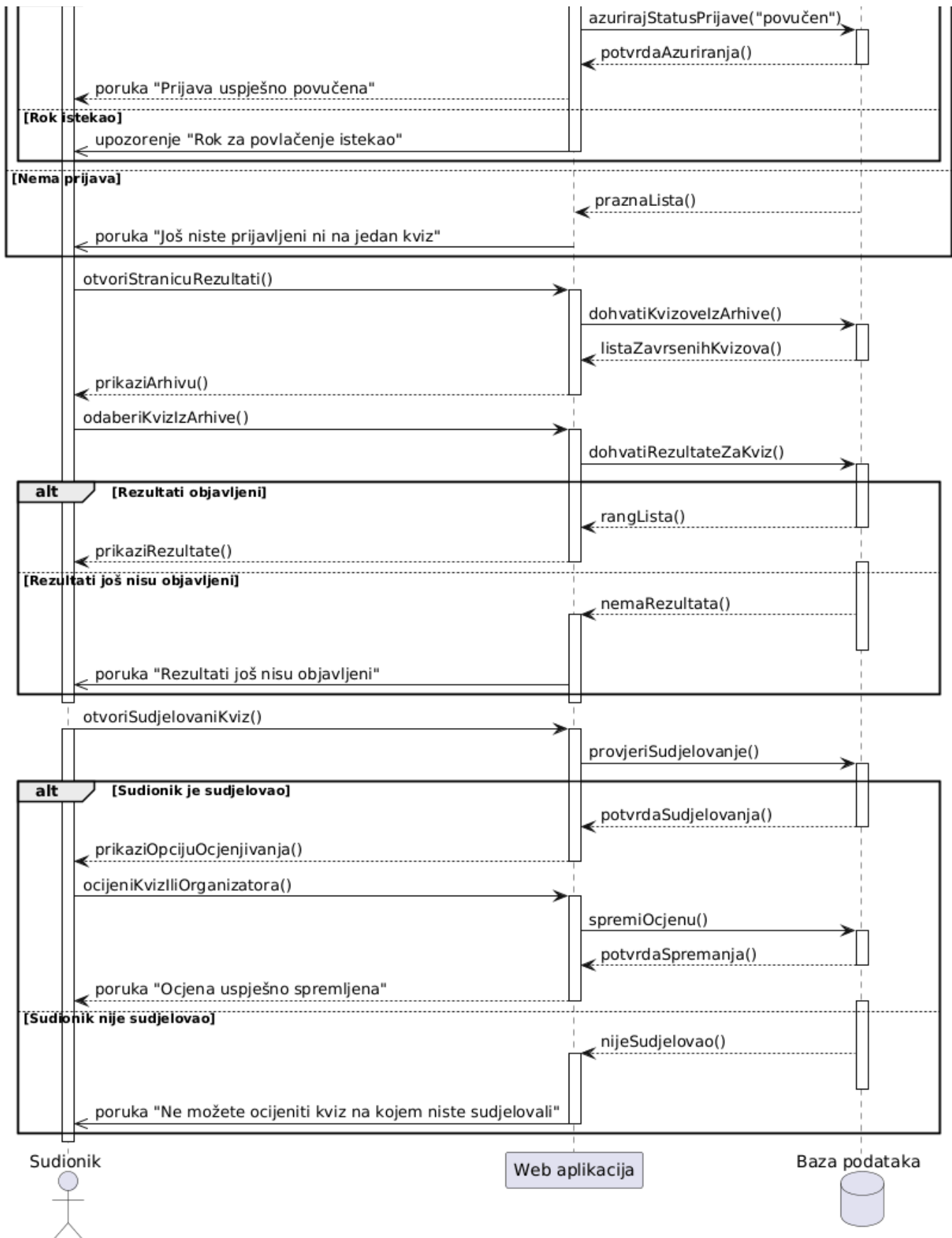




UC6, UC7, UC8, UC9, UC10, UC11 Sudionik - pretraživanje kvizeva, prijava na kviz, upravljanje svojim prijavama, pregled arhive završenih kvizeva, ocjenjivanje kviza/organizatora

Dijagram prikazuje procese koje korisnik, primarno sudionik, može izvršavati putem web aplikacije vezane uz pretragu i prijavu na kviz, upravljanje vlastitim prijavama, te pregled rezultata završenih kvizeva i ocjenjivanje.





UC12, UC13, UC14, UC15, UC17 Organizator - upravljanje kvizovima i prijavama, automatsko slanje obavijesti

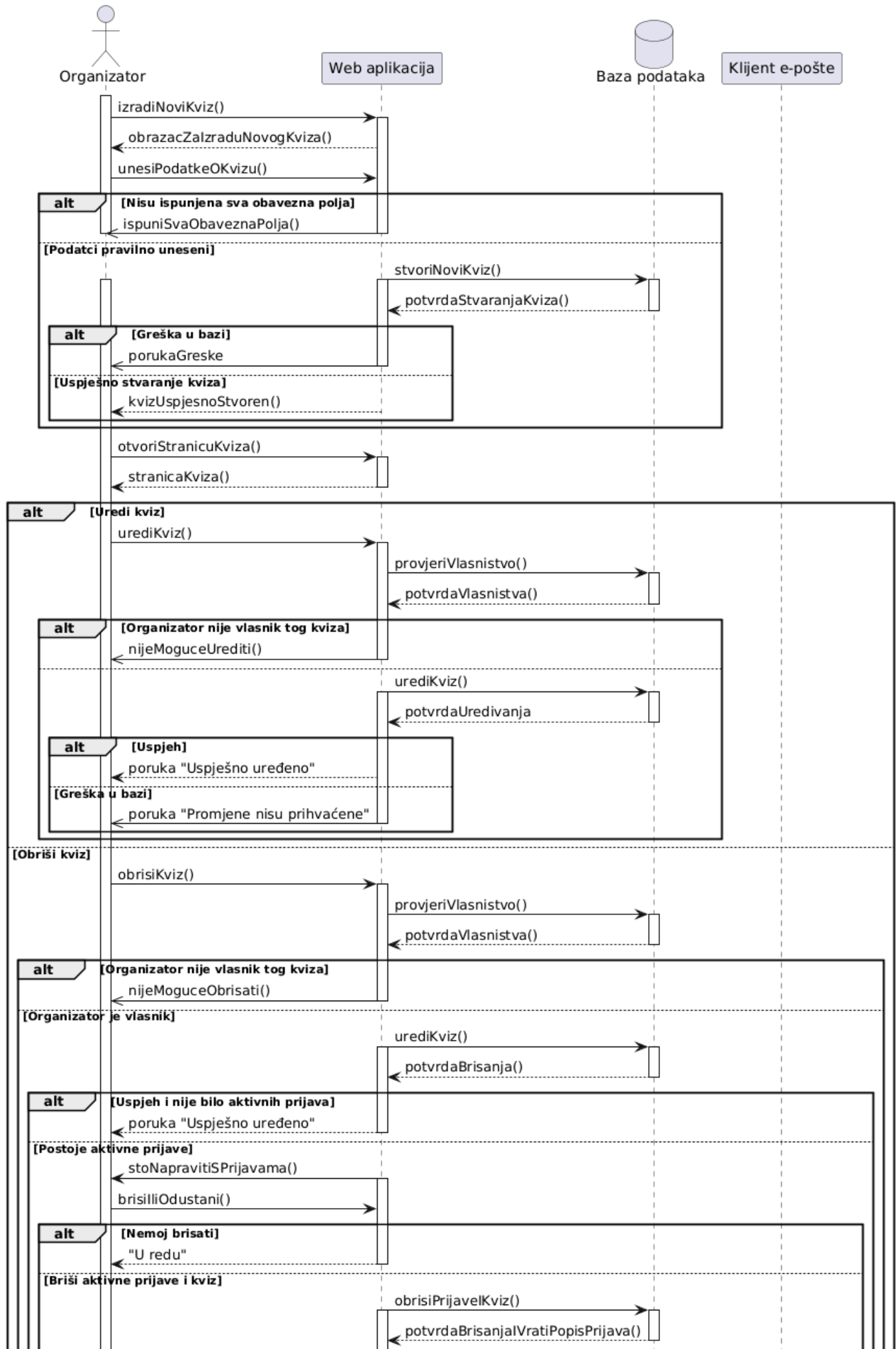
Dijagram prikazuje procese koje organizator može izvršavati putem web aplikacije u vezi s upravljanjem kvizovima. Organizator ima mogućnost stvaranja novog kviza, uređivanja i brisanja postojećih kvizova koje je sam kreirao, kao i upravljanja prijavama te unos rezultata nakon održanog kviza.

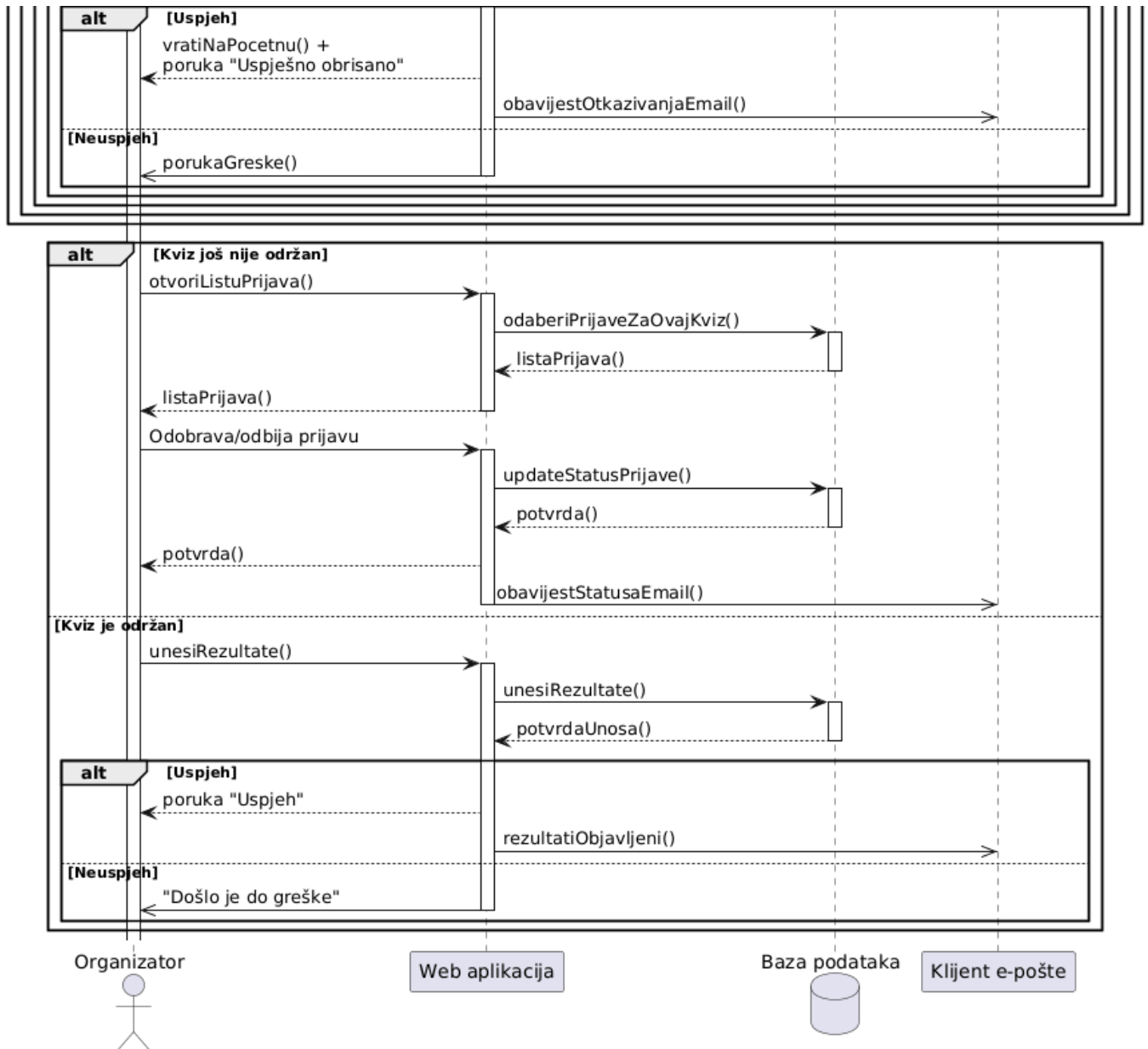
Proces započinje kada organizator pošalje zahtjev za izradu novog kviza. Aplikacija vraća obrazac za unos podataka, nakon čega organizator unosi potrebne informacije o kvizu. Ako nisu popunjena sva obavezna polja, sustav obavještava organizatora da ih mora ispuniti. U slučaju da su svi podaci ispravno uneseni, aplikacija šalje zahtjev bazi podataka za stvaranje novog kviza. Baza vraća potvrdu o uspješnom stvaranju kviza ili poruku o pogrešci ako dođe do problema.

Nakon stvaranja kviza, organizator može otvoriti stranicu kviza te odabrati želi li ga urediti ili obrisati. Ako odabere uređivanje, aplikacija prvo provjerava u bazi podataka je li organizator stvarni vlasnik kviza. Ako nije, sustav javlja da uređivanje nije moguće. Ako jest, baza se ažurira s novim podacima i, ovisno o ishodu, organizator dobiva poruku o uspješnom uređivanju ili o pogrešci.

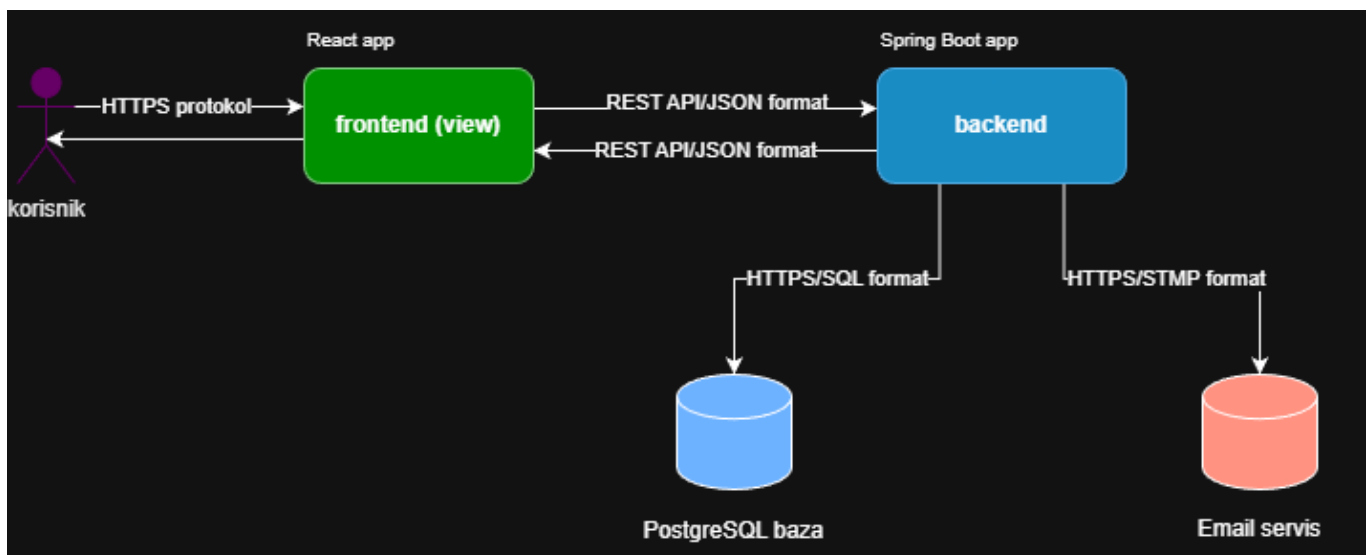
Ako organizator odluči obrisati kviz, također se provjerava vlasništvo. Ako je potvrđeno da je on vlasnik, sustav provjerava postoje li aktivne prijave. Ako ih nema, kviz se briše i prikazuje poruka o uspješnom brisanju. Ako postoje aktivne prijave, sustav traži od organizatora odluku želi li odustati ili obrisati i prijave i kviz. Ako odabere brisanje, baza podataka briše sve povezane prijave i kviz, vraća potvrdu te sustav šalje obavijest putem e-pošte prijavljenim korisnicima o otkazivanju kviza.

U dijelu koji se odnosi na prijave i rezultate, razlikuju se dvije situacije. Ako kviz još nije održan, organizator može pregledati listu prijava, odobriti ili odbiti sudionike te nakon potvrde izmjena sustav šalje e-mail obavijesti prijavljenima o statusu njihove prijave. Ako je kviz već održan, organizator unosi rezultate. Nakon što baza potvrdi unos, sustav prikazuje poruku o uspješnom unosu i šalje e-mail obavijest sudionicima o objavljenim rezultatima. U slučaju greške, organizator prima odgovarajuću poruku o neuspjelom unosu.





4. Arhitektura sustava



4.1 Opis arhitekture

4.1.1 Pregled arhitekture na visokoj razini

Sustav QuizWiz temelji se na višeslojnoj web arhitekturi klijent-poslužiteljskog tipa, implementiranoj kroz modularno organizirani sustav s jasnim odvajanjima slojeva (frontend, backend, baza podataka). Ovaj pristup omogućuje jednostavniju implementaciju i održavanje tijekom početne faze razvoja, uz mogućnost kasnije transformacije u arhitekturu mikroserversa ako se pojavi potreba za većom skalabilnošću.

4.1.2 Stil arhitekture

Odabrani arhitektonski stil je klijent-poslužitelj s MVC (Model-View-Controller) obrascem unutar backend sustava. Glavni razlozi za odabir ovog pristupa su: * jasno odvajanje odgovornosti između slojeva: korisničko sučelje, poslovna logika i podaci * jednostavna održivost i testiranje - promjene u logici ili prikazu mogu se provoditi neovisno * skalabilnost - backend se može proširiti u više servisa u budućnosti * kompatibilnost s web tehnologijama - arhitektura prirodno podržava REST API-je za komunikaciju s frontend slojem

Ovaj stil ima visoku koheziju i nisku povezanost, čime se postiže veća fleksibilnost, ponovna upotrebljivost i stabilnost sustava.

4.1.3 Podsustavi i njihove odgovornosti

podstav	opis i odgovornosti
autentifikacijski podstav	Upravlja registracijom, prijavom i autorizacijom korisnika. Koristi JWT (JSON Web Token) za autentifikaciju zahtjeva.
podstav upravljanja kvizovima	Omogućuje kreiranje, uređivanje, otkazivanje i pregled kvizova, uključujući unos informacija o datumu, mjestu, kapacitetu i oznakama (vrsta kviza).
podstav prijava timova	Omogućuje prijave timova na kvizove, odobravanje prijave od strane organizatora i automatsku kontrolu popunjenosti kapaciteta.

podstav	opis i odgovornosti
podstav notifikacija	Automatski šalje e-mail obavijesti o odobrenju, otkazivanju ili promjeni termina kviza.
administratorski podstav	Omogućuje administratoru pregled i upravljanje organizatorima, kvizovima i lokacijama.

4.1.4 Preslikavanje na radnu platformu

Sustav QuizWiz bit će implementiran na hibridnoj radnoj platformi, kombinirajući cloud infrastrukturu i mogućnost lokalnog razvoja. Odabrana platforma omogućuje fleksibilnost, skalabilnost i jednostavno održavanje sustava.

Predviđena implementacija: * frontend (React) - frontend aplikacija bit će deploy-ana na cloud platformu, što omogućuje dostupnost aplikacije korisnicima putem interneta * React aplikacija se razvija lokalno gdje se također provjerava rad aplikacije prije deploja u cloud (Vercel) * backend (Spring Boot) - backend aplikacija će biti deploy-ana na cloud server (Railway) * cloud deployment osigurava visoku dostupnost i otpornost na greške * baza podataka (PostgreSQL) - PostgreSQL baza će biti hostana u cloudu (Supabase) * cloud hostanje omogućuje automatsko skaliranje, sigurnosne kopije i monitoring performansi

Prednosti hibridnog pristupa: * fleksibilnost - razvojni tim može raditi lokalno, dok radna verzija radi u cloud okruženju * pouzdanost - cloud hostanje ima automatizirane backup-ove i monitoring, što smanjuje rizik od gubitka podataka i prekida rada sustava * razvoj i testiranje - lokalno okruženje omogućuje brze iteracije i debugging, a radna verzija za to vrijeme ostaje stabilna

4.1.5 Spremišta podataka

Sustav koristi relacijsku bazu podataka - PostgreSQL, zbog: * pogodnosti za strukturirane podatke poput korisnika, kvizova, timova i rezultata * podrške za složene odnose * integriteta podataka putem stranih ključeva * mogućnosti skaliranja i sigurnosti u cloud okruženju

4.1.6 Mrežni protokoli

- HTTPS - glavni protokol za komunikaciju između klijenta i poslužitelja putem REST API poziva
- WebSocket (u kasnijim fazama) - za real-time notifikacije o promjenama kvizova i statusima prijava
- SMTP - za slanje e-mail obavijesti putem vanjskih servera

4.1.7 Globalni upravljački tok

1. Korisnik (organizator ili igrač) pristupa web aplikaciji putem preglednika (frontend).
2. Frontend šalje zahtjev REST API-ju (backend) putem HTTPS protokola.
3. Backend obrađuje zahtjev odgovarajućim kontrolerom i komunicira s bazom podataka.
4. Nakon obrade, backend vraća odgovor u JSON formatu.
5. Frontend prikazuje rezultat korisniku i, prema potrebi, pokreće notifikacijski podsustav.
6. U slučaju izmjena (npr. otkazivanja kviza), backend automatski šalje obavijesti putem e-maila svim prijavljenim korisnicima.

4.1.8 Sklopovskoprogramski zahtjevi

komponenta	minimalni zahtjev
operativni sustav (OS)	Linux, Windows ili macOS
procesor	dvojezgreni CPU (2.0GHz+)
RAM memorija	4GB (razvojno), 8GB (poslužiteljsko okruženje)
preglednik	Chrome, Edge, Firefox, Brave, Safari...
okruženja	Node.js (v18+), PostgreSQL (v15+)

4.1.9 Upravljanje autentifikacijom i sesijama putem JWT-a

U svrhu sigurne autentifikacije korisnika i zaštite API poziva, sustav koristi JWT (JSON Web Token) mehanizam za upravljanje sesijama. Ovaj pristup omogućuje stateless autentifikaciju, eliminirajući potrebu za pohranom sesijskih podataka na strani poslužitelja, što pridonosi skalabilnosti i sigurnosti sustava.

Mehanizam rada:

- Prilikom prijave, poslužitelj (Spring Boot backend) generira JWT koji uključuje identitet korisnika, ulogu i vrijeme isteka.
- Token se digitalno potpisuje tajnim ključem te vraća klijentskoj aplikaciji (frontend – React).
- Frontend pohranjuje token (npr. u `localStorage`) te ga šalje u svakom sljedećem zahtjevu prema backendu u HTTPS zaglavlju.
- Backend validira token prije obrade svakog zahtjeva i, u slučaju valjanosti, omogućuje pristup zaštićenim resursima.

Prednosti:

- **Skalabilnost** – stateless pristup omogućuje jednostavno horizontalno skaliranje backend aplikacije.
- **Sigurnost** – digitalno potpisani tokeni štite od manipulacije i smanjuju napade poput CSRF-a.

- **Jednostavno upravljanje sesijama** – frontend jasno razlikuje autentificirano stanje bez potrebe za server-side sesijama.

Sigurnosne napomene:

- Tokeni imaju definirani rok trajanja (exp claim) i automatski istječu nakon određenog vremena.
- Planira se podrška za **refresh token** mehanizam radi produljenja trajanja sesije bez ponovne prijave.
- Komunikacija između frontend-a i backend-a odvija se isključivo putem HTTPS protokola.

JWT pristup u potpunosti je usklađen s višeslojnom web arhitekturom i pridonosi boljoj modularnosti, sigurnosti i neovisnom razvoju frontend i backend komponenti.

4.2 Obrazloženje odabira arhitekture

4.2.1 Izbor arhitekture temeljen na principima oblikovanja

Glavni principi korišteni pri dizajnu: * visoka kohezija - komponente imaju jasno definirane funkcije * niska povezanost - frontend i backend su neovisni i komuniciraju isključivo putem REST API-ja * fleksibilnost - modularna struktura koda omogućuje dodavanje novih funkcionalnosti bez većih promjena * sigurnost - autentifikacija i autorizacija putem JWT tokena te HTTPS komunikacija * skalabilnost - mogućnost prelaska s monolitne na mikrouslužnu arhitekturu u budućnosti

4.2.2 Razmatrane alternative

Mikrouslužna arhitektura je bila razmatrana, ali je odbačena zbog nepotrebne složenosti u početnoj fazi projekta. NoSQL baza je razmotrena, ali je odbačena jer aplikacija koristi čvrste relacijske veze i zahtijeva integritet podataka. Serverless pristup je također razmatran, no trenutno nije potreban zbog malog broja istovremenih korisnika.

4.3 Organizacija sustava na visokoj razini

- Klijent-poslužitelj: Frontend (React) i backend (Spring Boot) komuniciraju putem REST API-ja. Klijent se spaja na backend preko HTTPS protokola i dohvaća podatke u JSON formatu.
- Baza podataka: Relacijska baza PostgreSQL pohranjuje sve korisničke, kvizne i prijavne podatke, uz definirane odnose i indeksiranje za brze pretrage.
- Datotečni sustav: Korisnički avatari i slike kvizova pohranjuju se u cloud spremište.

- Grafičko sučelje: Frontend koristi React za responzivno, intuitivno web sučelje. Komunikacija s backendom odvija se putem REST API poziva.

4.4 Organizacija aplikacije

4.4.1 Frontend i Backend slojevi

Sustav QuizWiz organiziran je u dva glavna dijela: frontend i backend, koji zajedno čine višeslojnu web arhitekturu prema MVC obrascu. Komunikacija između slojeva odvija se putem REST API-ja, a podaci se razmjenjuju u JSON formatu.

Frontend, implementiran u React.js, čini prezentacijski (view) sloj sustava. On omogućuje korisnicima interakciju s aplikacijom te vizualni prikaz podataka dohvaćenih s backend-a.

Glavne značajke: * Struktura se temelji na modularnim i višekratno upotrebljivim komponentama (QuizList, LoginForm). * Komunikacija s backendom odvija se putem HTTPS zahtjeva (GET, POST...). * Obrada i prikaz odgovora je u JSON formatu. * Upravljanje stanjem i navigacijom obavlja se pomoću React hookova i routera.

Backend, implementiran u Spring Boot frameworku, čini poslužiteljski (server) dio sustava i implementira MVC (Model-View-Controller) arhitekturu. Njegova glavna zadaća je obrada zahtjeva primljenih od frontenda, izvršavanje poslovne logike, pristup bazi podataka te vraćanje odgovora u JSON formatu.

4.4.2 MVC arhitektura

Organizacija slojeva unutar backenda-a:

sloj	Opis i odgovornosti
Routes/API endpointovi	Definiraju REST API putanje koje primaju zahtjeve s frontenda.
Controller	Prima i obrađuje HTTPS zahtjeve, poziva odgovarajuće servise te vraća JSON odgovore. Odgovoran je za povezivanje frontend-a s poslovnom logikom.
Service	Implementira poslovnu logiku - autentifikaciju i autorizaciju korisnika, slanje e-mail notifikacija, upravljanje kvizovima, timovima i korisnicima.

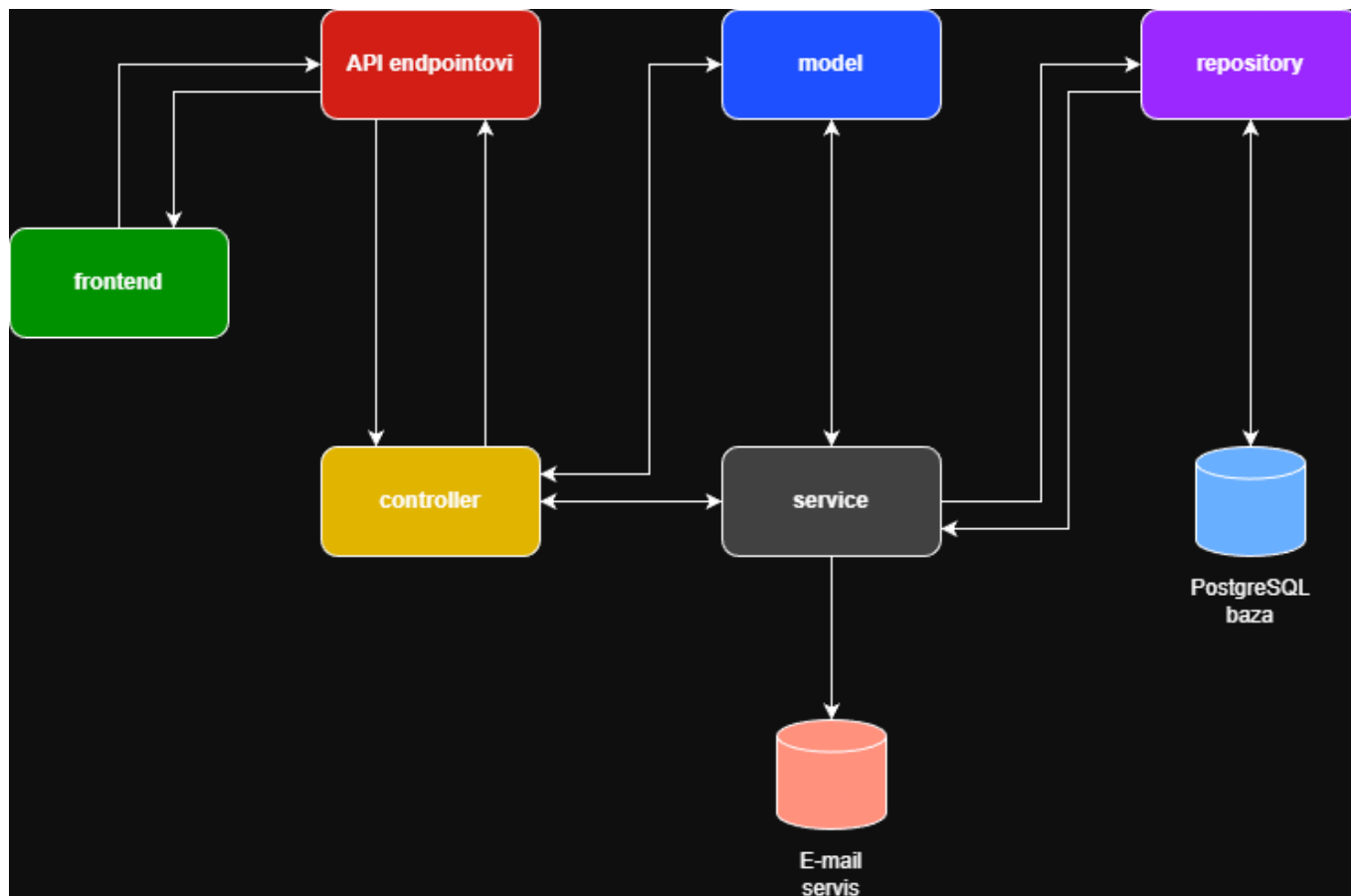
sloj**Opis i odgovornosti**

Repository

Komunicira s bazom podataka koristeći Supabase API.

Model

Predstavlja podatkovni sloj, odnosno Java entitete koji preslikavaju tablice u PostgreSQL bazi. Svaki entitet definira strukturu podataka.



Baza podataka

4.5 Opis tablica

4.5.1 KORISNIK

Atribut

Tip podatka

**Opis
varijable**

Korisnik_id

INT

Jedinstveni
identifikator
korisnika

Atribut	Tip podatka	Opis varijable
Username	VARCHAR (50)	Korisničko ime
Email	VARCHAR (100)	Adresa e-pošte korisnika
Password	VARCHAR (255)	Lozinka korisnika
Kontakt_broj	VARCHAR (20)	Telefonski broj korisnika
Uloga	ENUM('IGRAČ', 'ORGANIZATOR')	Uloga korisnika u sustavu

4.5.2 IGRAČ

Atribut	Tip podatka	Opis varijable
Korisnik_id (FK)	INT	Referenca na tablicu KORISNIK

4.5.3 ORGANIZATOR

Atribut	Tip podatka	Opis varijable
Organizator_id	INT	Jedinstveni identifikator organizatora
Korisnik_id (FK)	INT	Referenca na tablicu KORISNIK

4.5.4 TIM

Atribut	Tip podatka	Opis varijable
Team_id	INT	Jedinstveni identifikator tima
Ime_tima	VARCHAR (100)	Naziv tima
Broj_clanova	INT	Broj članova tima
Status_prijave	ENUM ('ČEKA', 'ODOBRENO', 'ODBIJENO')	Status prijave tima
BrBodova	INT	Broj osvojenih bodova
Rang	INT	Trenutni rang tima
sent_at	DATETIME	Datum i vrijeme slanja prijave
odobreno_at	DATETIME	Datum i vrijeme odobrenja prijave
Kviz_id (FK)	INT	Referenca na tablicu KVIZ

4.5.5 KVIZ

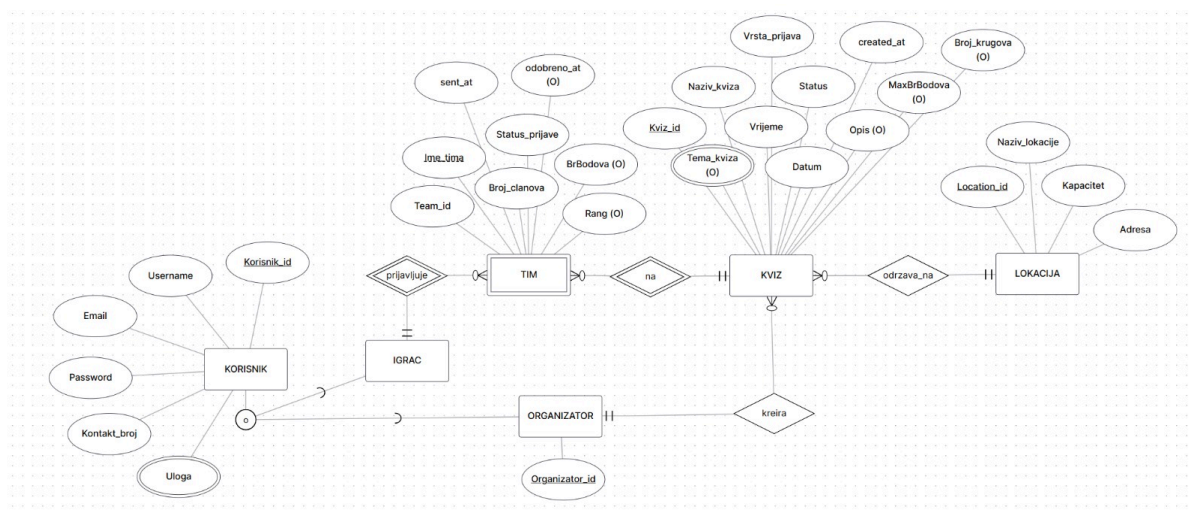
Atribut	Tip podatka	Opis varijable
Kviz_id	INT	Jedinstveni identifikator kviza
Naziv_kviza	VARCHAR (100)	Naziv kviza
Tema_kviza	VARCHAR (100)	Tema kviza
Datum	DATE	Datum održavanja kviza
Vrijeme	TIME	Vrijeme održavanja kviza

Atribut	Tip podatka	Opis varijable
Vrsta_prijava	ENUM ('TIMSKA', 'POJEDINAČNA')	Tip prijave
Status	ENUM ('AKTIVAN', 'ZAVRŠEN', 'OTKAZAN')	Status kviza
Opis	TEXT	Kratki opis kviza
created_at	DATETIME	Datum i vrijeme kreiranja kviza
MaxBrBodova	INT	Maksimalni broj bodova
Broj_krugova	INT	Ukupan broj krugova u kvizu
Location_id (FK)	INT	Referenca na tablicu LOKACIJA
Organizator_id (FK)	INT	Referenca na tablicu ORGANIZATOR

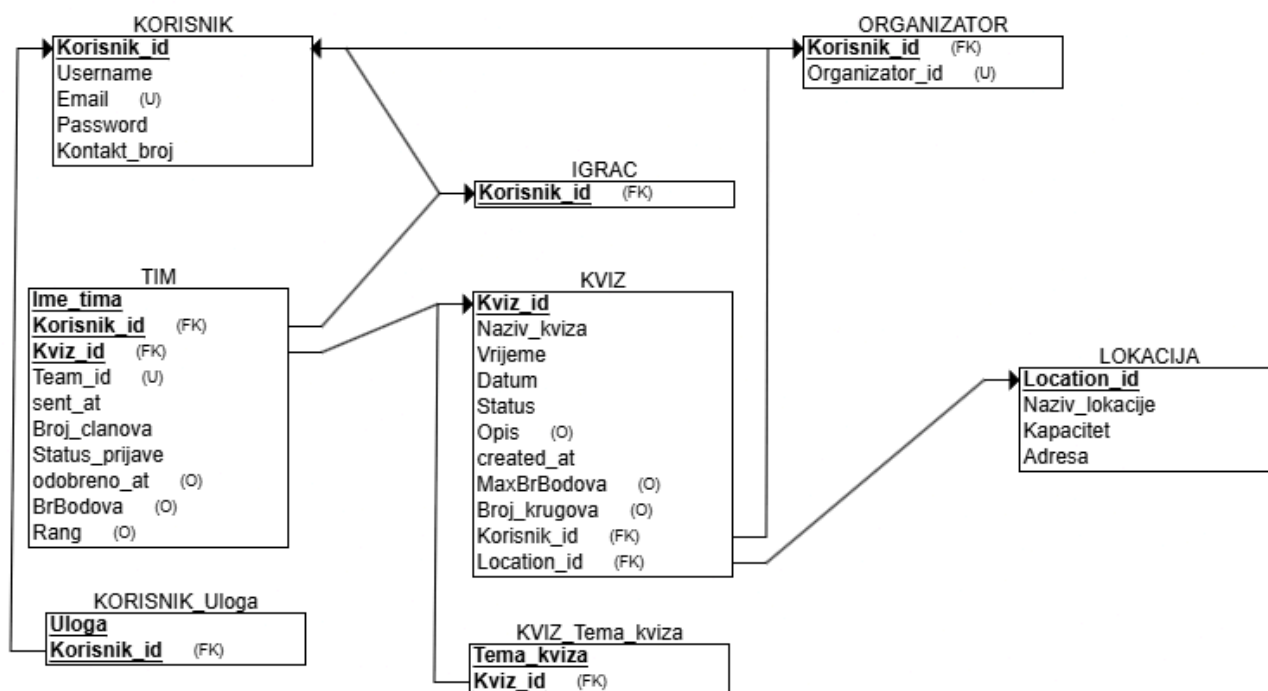
4.5.6 LOKACIJA

Atribut	Tip podatka	Opis varijable
Location_id	INT	Jedinstveni identifikator lokacije
Naziv_lokacije	VARCHAR (100)	Naziv lokacije kviza
Kapacitet	INT	Maksimalan broj sudionika
Adresa	VARCHAR (255)	Adresa održavanja kviza

4.6 Dijagram baze podataka



Image



4.7 Dijagram razreda

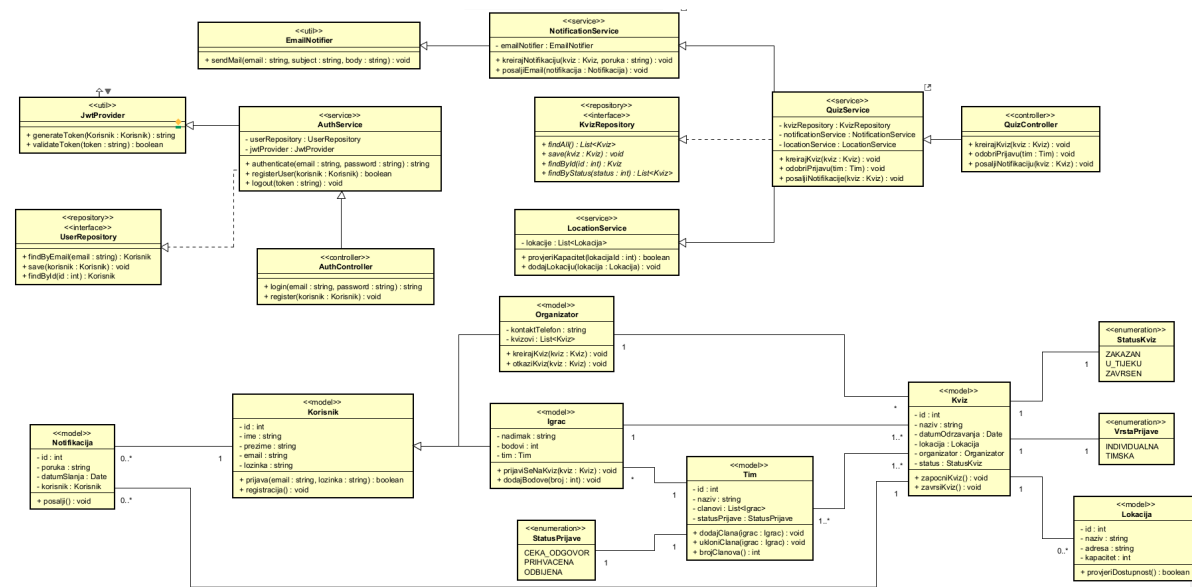
Ovaj dijagram prikazuje strukturu i odnose između razreda uključenih u sustav ‘QuizWiz’, koji omogućuje prijavu korisnika, organizaciju kvizova i slanje obavijesti sudionicima.

Korisnici su predstavljeni klasom ‘Korisnik’, iz koje nasljeđuju razredi ‘Igrač’ i ‘Organizator’, čime se omogućuje razlikovanje uloga u sustavu. Klasa ‘AuthController’ upravlja autentifikacijom i registracijom korisnika putem ‘AuthService’, koji koristi ‘UserRepository’ za pristup podacima o korisnicima te ‘JwtProvider’ za generiranje sigurnosnih tokena.

Organizatori mogu stvarati i upravljati kvizovima preko klase ‘QuizController’, koja se oslanja na ‘QuizService’. Taj servis koristi ‘KvizRepository’ za rad s podacima o kvizovima, ‘LocationService’ za provjeru dostupnosti lokacija i ‘NotificationService’ za slanje obavijesti. Klasa ‘NotificationService’ koristi pomoćnu klasu ‘EmailNotifier’ za slanje poruka korisnicima.

U modelu su definirani razredi ‘Kviz’, ‘Lokacija’, ‘Tim’ i ‘Notifikacija’, koji međusobno povezuju korisnike, lokacije i statuse kvizova. Enum razredi ‘StatusPrijave’, ‘StatusKviz’ i ‘VrstaPrijave’ određuju stanja i vrste prijava unutar sustava.

Ovaj dijagram jasno prikazuje slojevit arhitekturu sustava, s odvojenim kontrolerima, servisima, repozitorijima, modelima i pomoćnim klasama, što doprinosi preglednosti i modularnosti aplikacije.

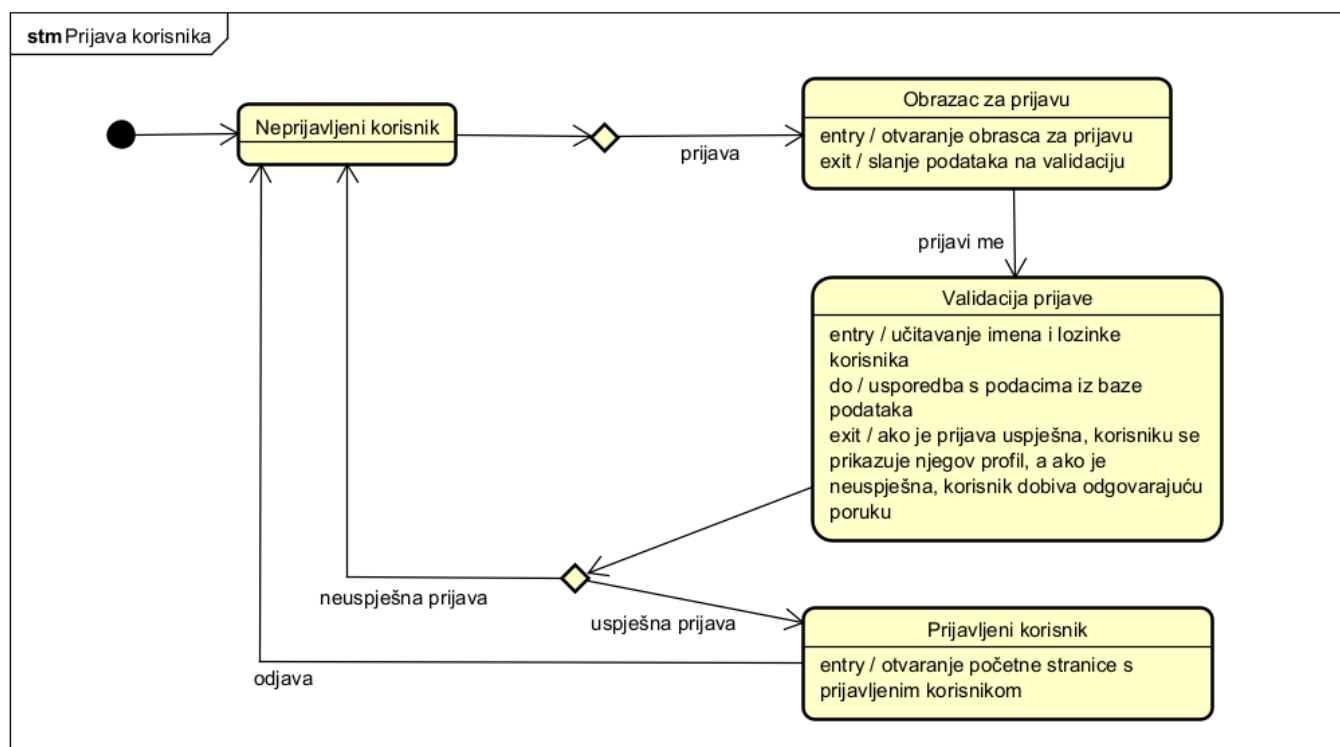


image

4.8 Dinamičko ponašanje aplikacije

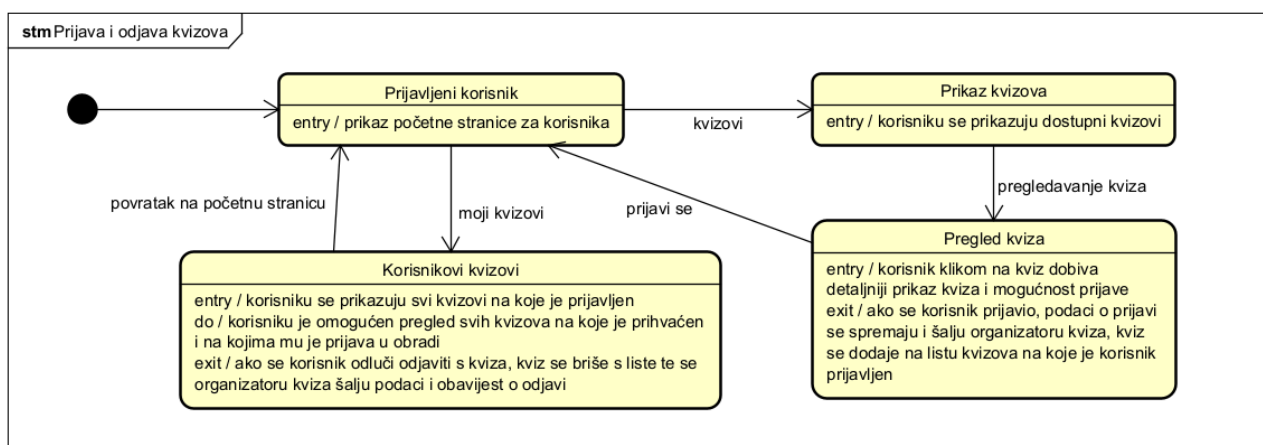
4.8.1 Prijava korisnika

Neprijavljeni korisnik na početnoj stranici može birati između prijave i registracije. U slučaju da odabere registraciju, otvara se obrazac za registraciju koji korisnik mora ispuniti pazeći pritom da prije pritiska tipke “Registriraj me” popuni sva obavezna polja. Pritiskom tipke “Registriraj me” uneseni podaci šalju se u bazu te korisnik dobiva potvrdu o registraciji. Ako neprijavljeni korisnik odabere prijavu, otvara se obrazac za prijavu koju korisnik mora popuniti u potpunosti. Pritiskom na gumb “Prijavi me” podaci se šalju na validaciju. Ako postoji korisnički račun koji odgovara unesenim podacima, korisnik se prijavljuje u sustav i otvara mu se početna stranica za njegovu ulogu. Ako ne postoji takav račun, korisniku se ispisuje obavijest o neuspješnoj prijavi.



4.8.2 Prijava i odjava kvizova

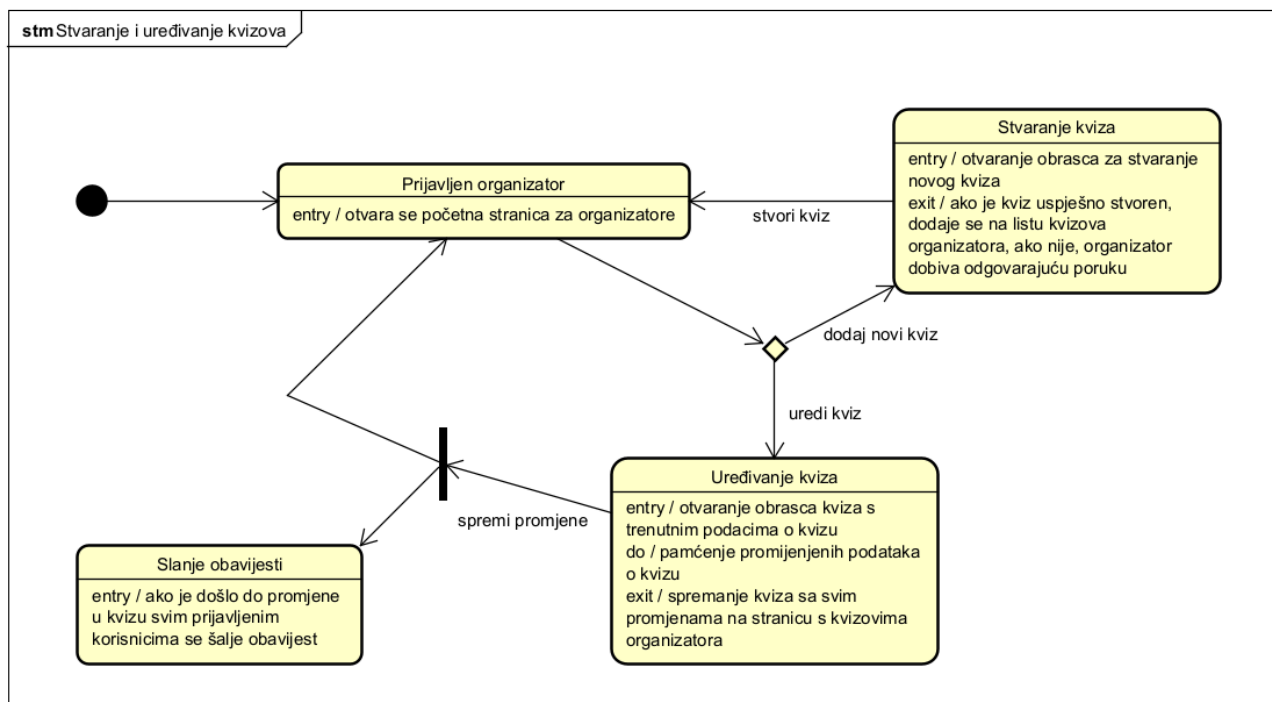
Prijavljeni korisnik ima opciju pregleda svih dostupnih kvizova i kvizova na koje je on prijavljen. Ako se želi prijaviti na određeni kviz, pri pregledu kvizova može se prijaviti klikom na gumb “Prijavi se”. Time se taj kviz dodaje na listu kvizova na koje je korisnik prijavljen. Korisnik može pregledavati sve svoje kvizove te u slučaju da se predomislio oko sudjelovanja, može se odjaviti s kviza.



4.8.3 Stvaranje i uređivanje kvizova

Prijavljen organizator ima opcije dodavanja novih i uređivanja postojećih kvizova. Odabirom dodavanja novog kviza otvara se obrazac za unos podataka o kvizu. Uspješnim unosom svih podataka i klikom na gumb “Stvori kviz” stvara se novi kviz i dodaje se na popis kvizova organizatora. Na popisu svih kvizova organizator ima mogućnosti uređivanja kvizova. Klikom na gumb “Uredi kviz” otvara se obrazac s već

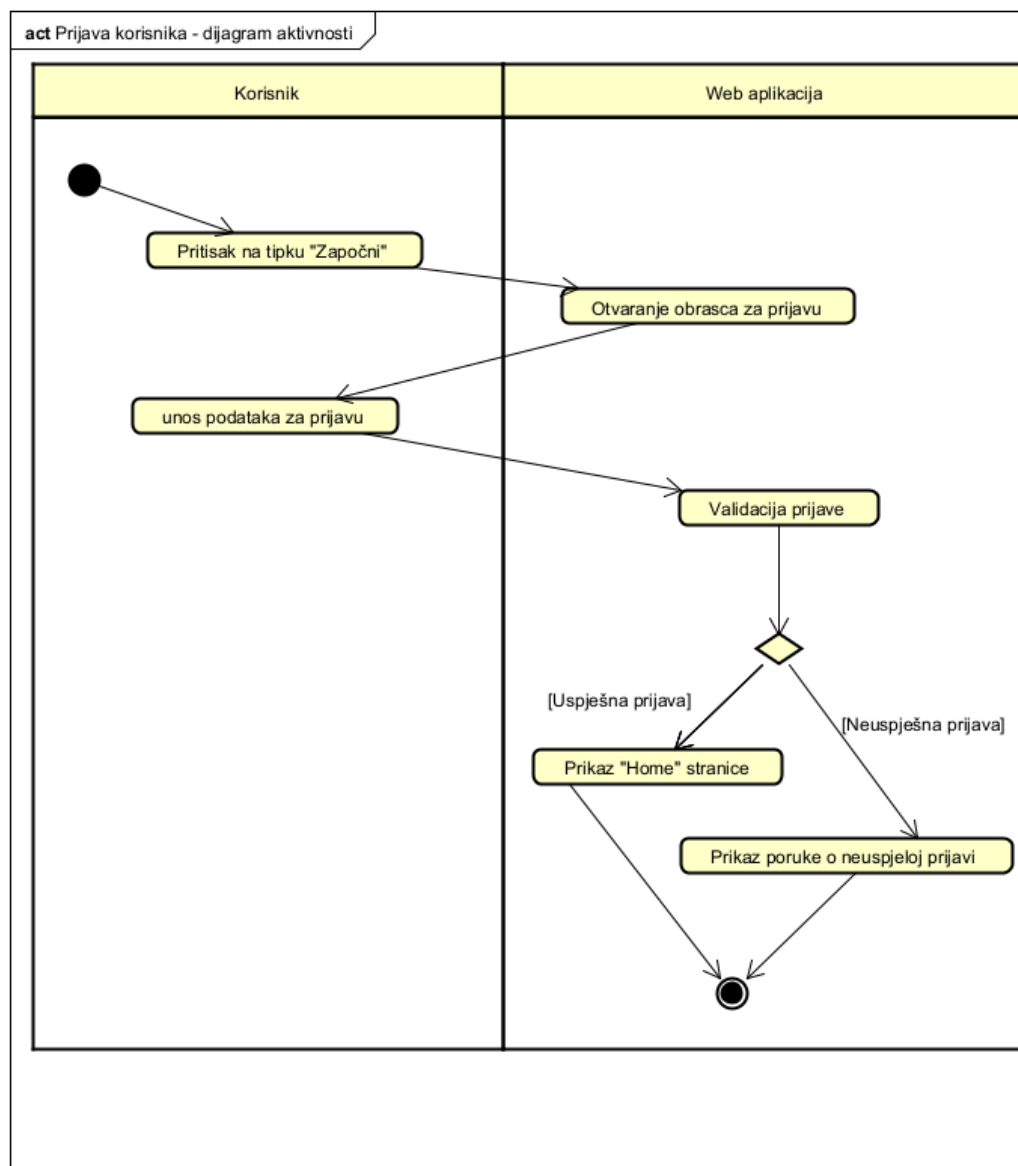
popunjenim detaljima kviza koje onda organizator može mijenjati. Ako je došlo do promjena, svim prijavljenim sudionicima šalje se obavijest da je došlo do promjena.



4.9 UML dijagrami aktivnosti

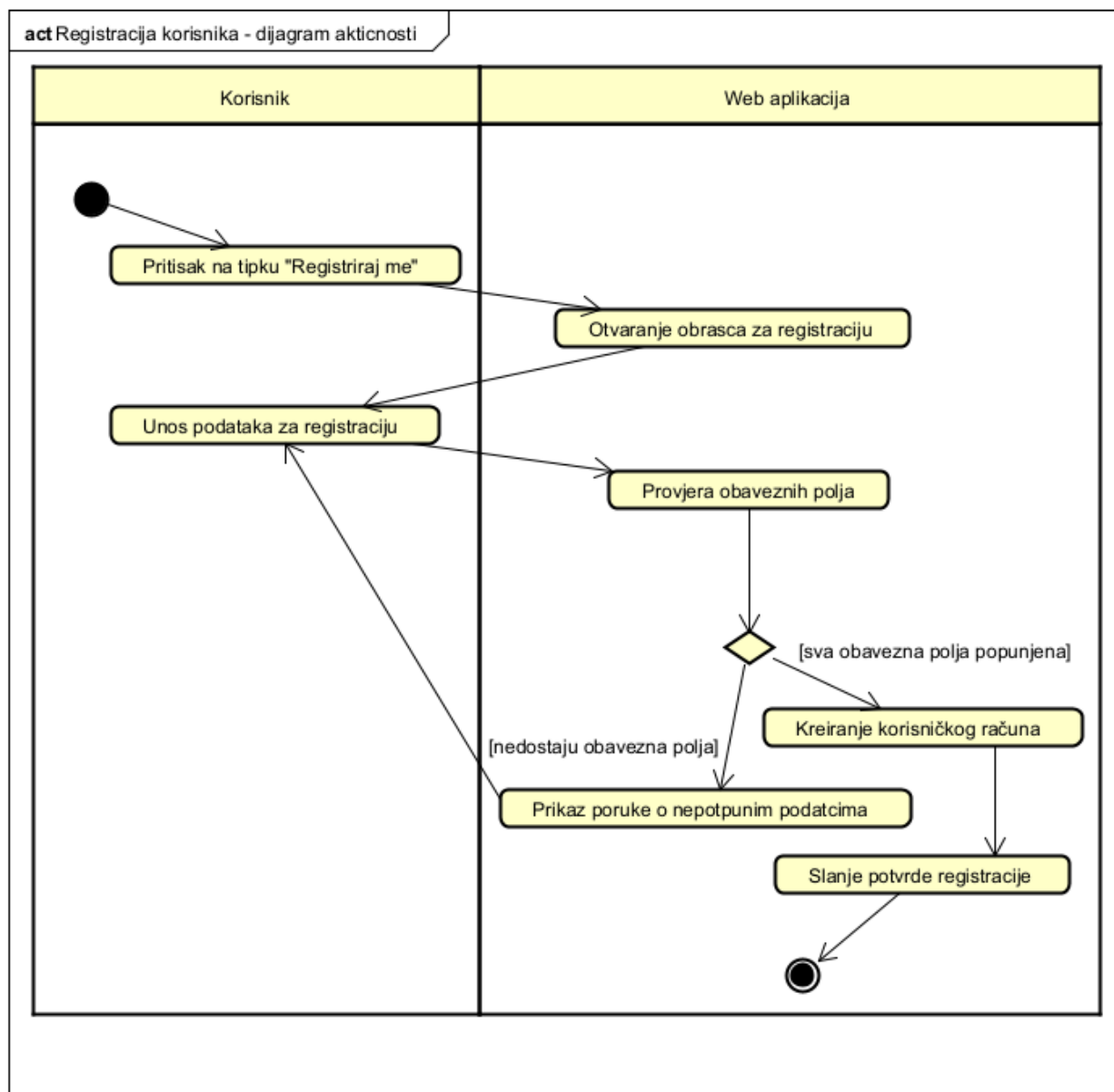
4.9.1 Prijava korisnika

Dijagram aktivnosti prikazuje proces prijave korisnika u sustav. Proces započinje pritiskom tipke „Započni“ od strane korisnika, nakon čega se otvara obrazac za prijavu. Korisnik zatim unosi prijavne podatke, a sustav provodi njihovu validaciju. Ako su podaci ispravni i postoji odgovarajući korisnički račun, korisniku se prikazuje početna stranica prijavljenog korisnika. U suprotnom, sustav prikazuje poruku o neuspješnoj prijavi te se proces završava.



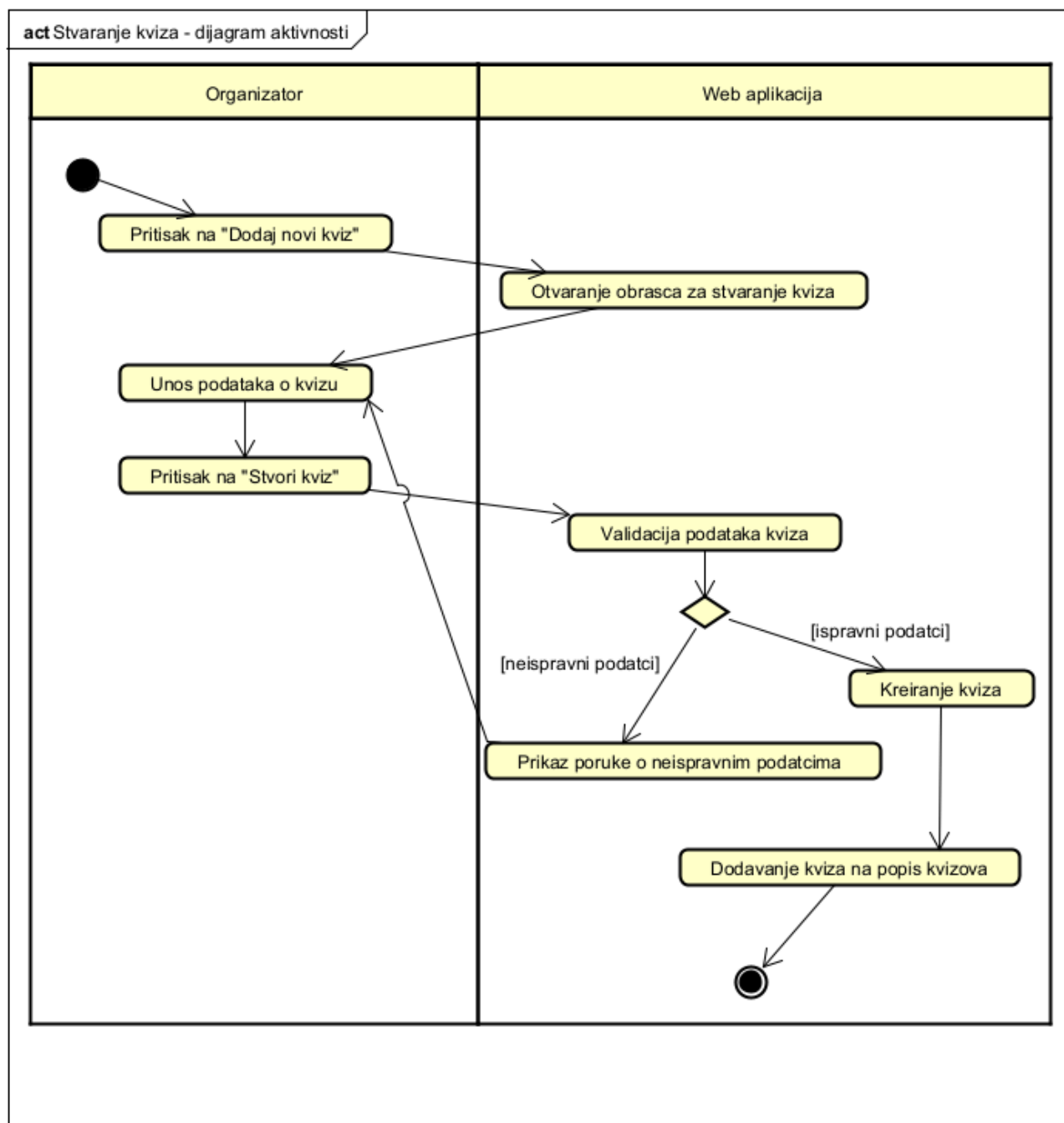
4.9.2 Registracija korisnika

Dijagram aktivnosti prikazuje proces registracije korisnika u web aplikaciju. Proces započinje pritiskom tipke „Registriraj me“ od strane korisnika, nakon čega web aplikacija otvara obrazac za registraciju. Korisnik zatim unosi registracijske podatke, a web aplikacija provjerava jesu li popunjena sva obavezna polja. Ako neka od obaveznih polja nisu popunjena, web aplikacija prikazuje poruku o nepotpunim podacima te omogućuje korisniku ponovni unos registracijskih podataka. Ako su sva obavezna polja ispravno popunjena, web aplikacija kreira korisnički račun i šalje potvrdu registracije, nakon čega se proces završava.



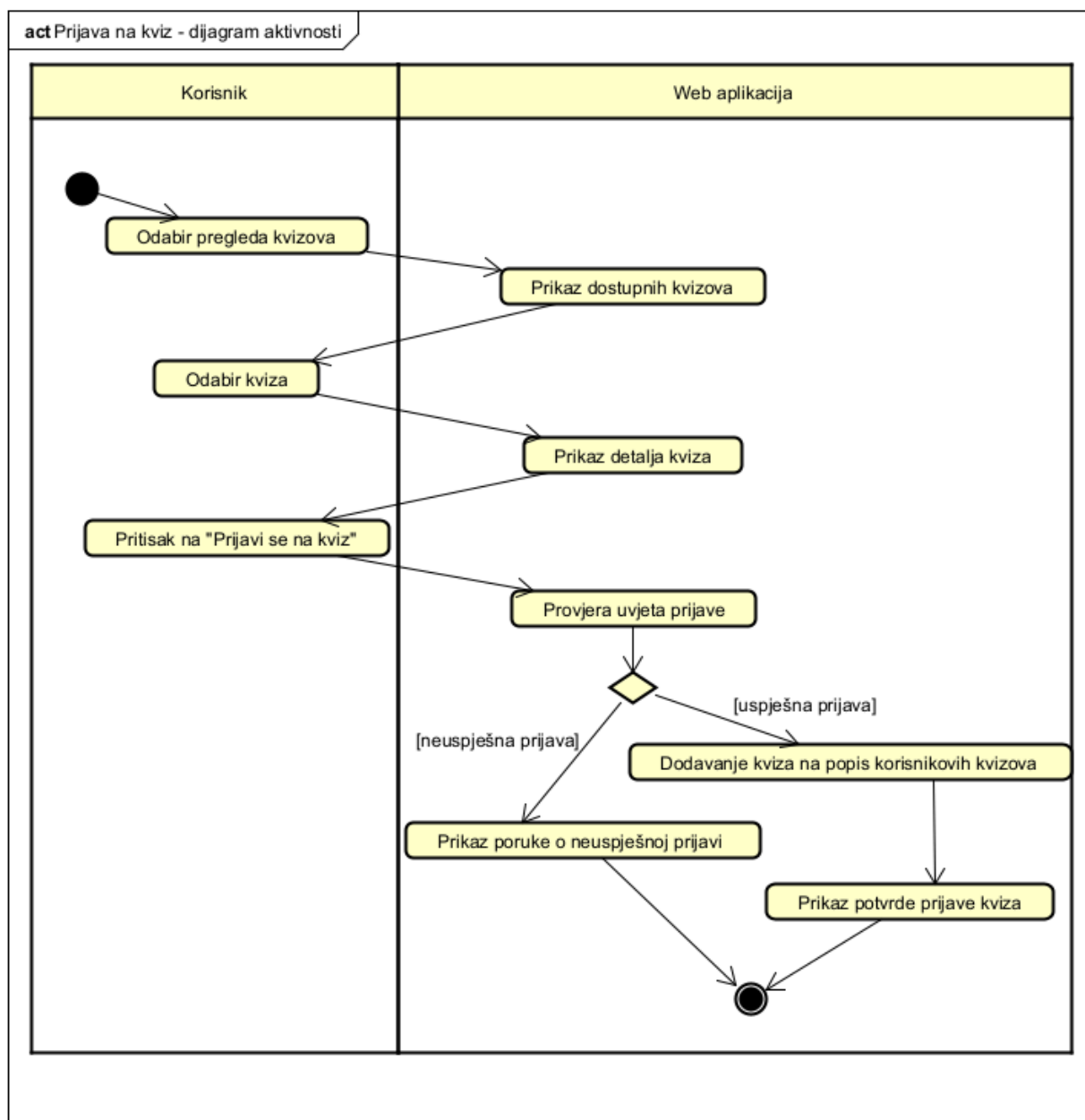
4.9.3 Stvaranje kviza

Dijagram aktivnosti prikazuje proces stvaranja novog kviza od strane prijavljenog organizatora. Proces započinje pritiskom na opciju „Dodaj novi kviz“, nakon čega web aplikacija otvara obrazac za stvaranje kviza. Organizator unosi podatke o kvizu i potvrđuje unos klikom na gumb „Stvori kviz“. Web aplikacija zatim provodi validaciju unesenih podataka. Ako su podaci neispravni, prikazuje se poruka o neispravnim podacima te se organizator vraća na unos podataka o kvizu. Ako su podaci ispravni, web aplikacija kreira novi kviz i dodaje ga na popis kvizova organizatora, nakon čega se proces završava.



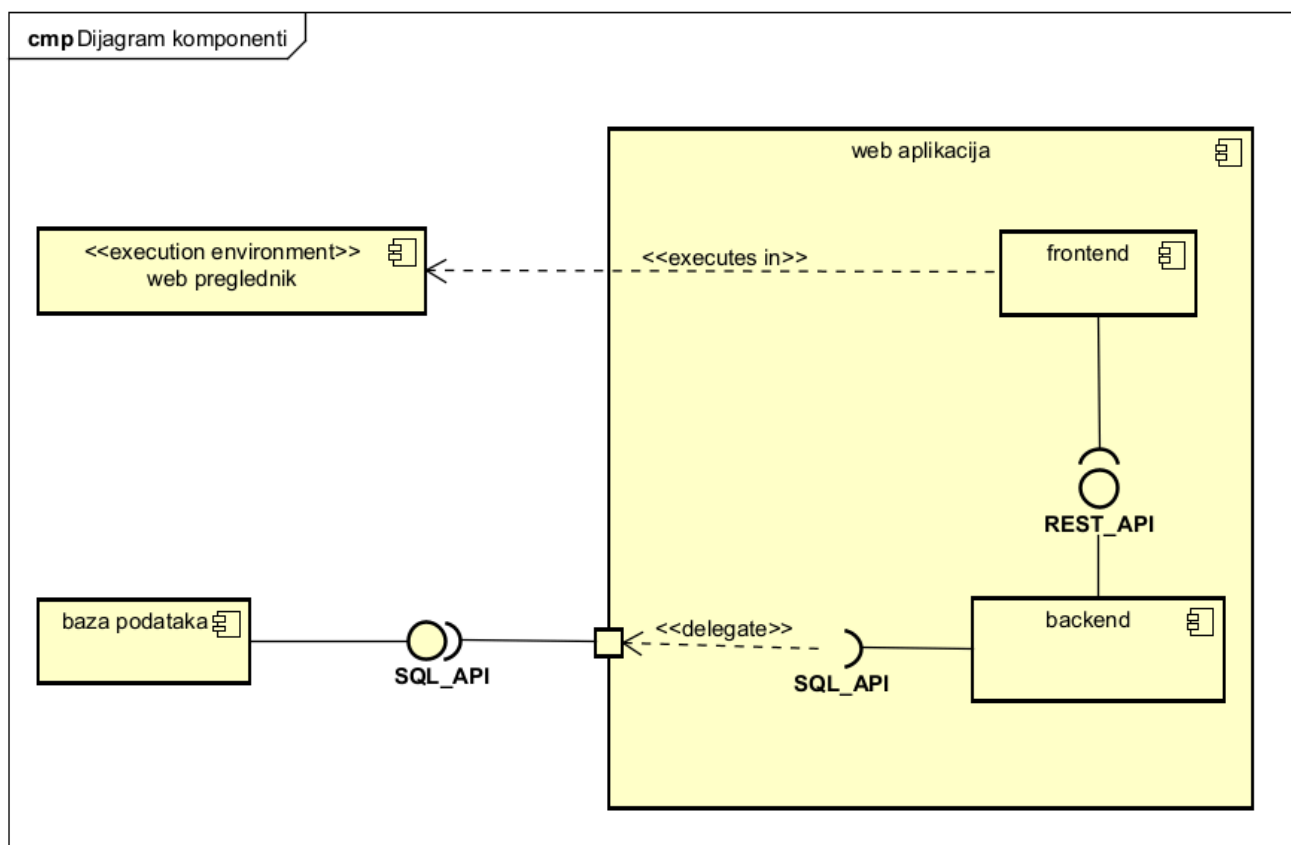
4.9.4 Prijava na kviz

Dijagram aktivnosti prikazuje proces prijave korisnika na odabrani kviz. Proces započinje odabirom pregleda dostupnih kvizova, nakon čega web aplikacija prikazuje popis dostupnih kvizova. Korisnik odabire kviz te mu se prikazuju detalji kviza i mogućnost prijave. Klikom na gumb „Prijavi se na kviz“ web aplikacija provjerava uvjete prijave. Ako je prijava uspješna, kviz se dodaje na popis korisnikovih kvizova te se korisniku prikazuje potvrda uspješne prijave. U suprotnom, web aplikacija prikazuje poruku o neuspješnoj prijavi te se proces završava.



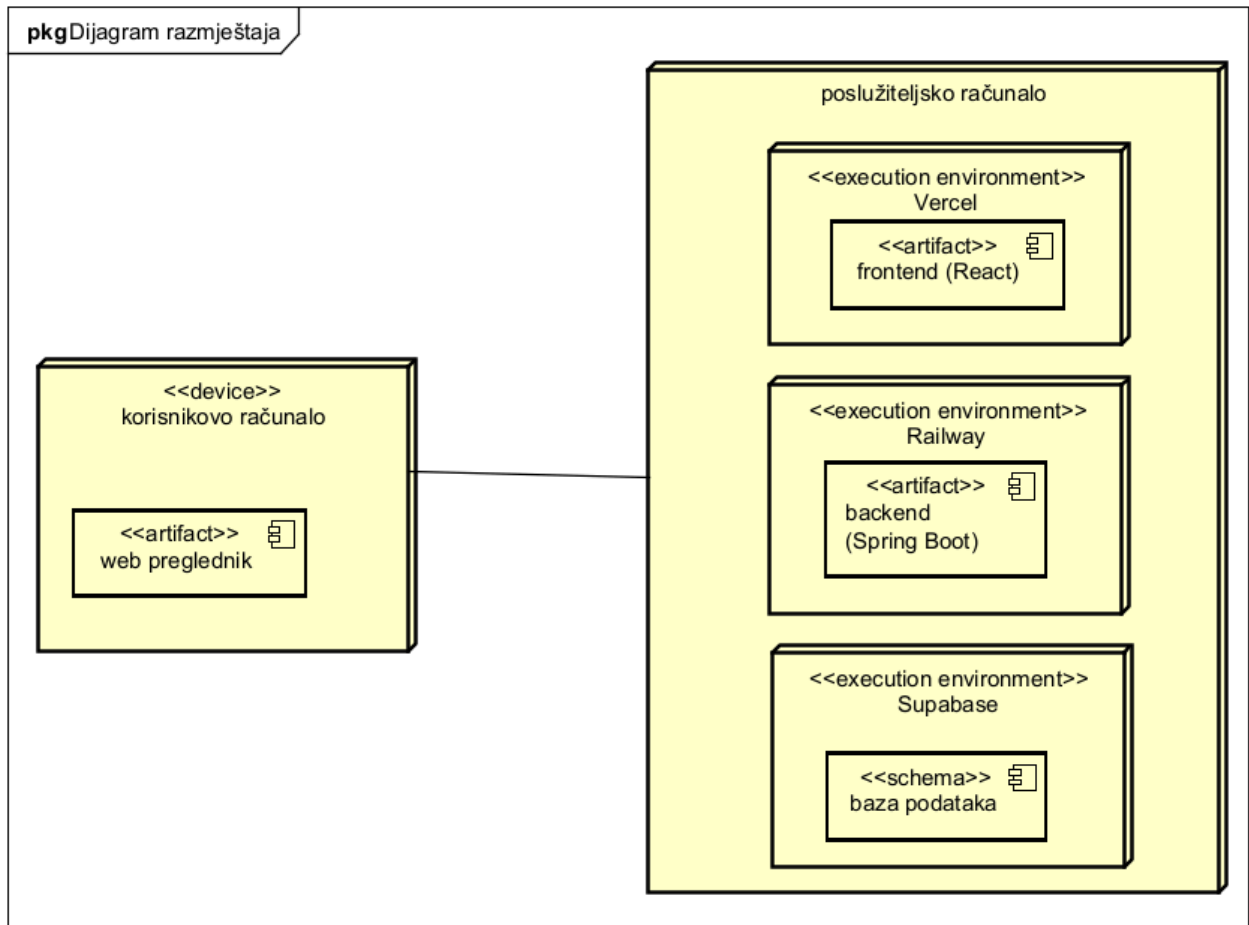
5.1 Dijagram komponenti

Dijagram komponenti prikazuje komponente našeg sustava, njihovu povezanost i način na koji komuniciraju. Sustav se sastoji od komponenti: web preglednik, frontend, backend i baza podataka. Pri pokretanju web stranice naše aplikacije, odmah se otvara web aplikacija. Frontend i backend komuniciraju putem REST API-ja, a backend i baza podataka putem SQL API-ja.



5.2 Dijagram razmještaja

Dijagram razmještaja prikazuje gdje se određeni dijelovi sustava nalaze, u našem slučaju korisnik pristupa aplikaciji putem web preglednika na vlastitom računalu, dok se na poslužitelju nalaze aplikacija i baza podataka, klijent i poslužitelj međusobno komuniciraju REST API-em. Frontend je deployan na Vercel, backend na Railway, a baza podataka na Supabase.

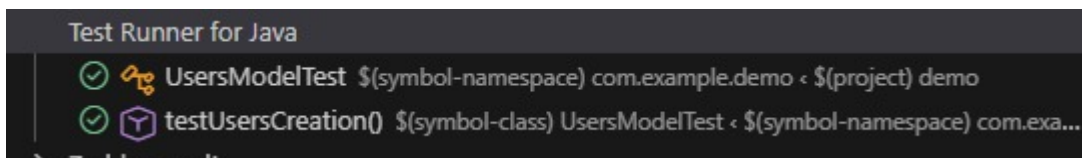


6.1 Ispitivanje komponenti

6.1.1 redovni slučaj

1. Testiranje gettera i settera za Quiz
2. Ispitni slučaj:

- ulazni podaci:
 - quizName: “Kviz znanja”
 - quizTheme: “Opce znanje”
 - applicationType: “Standard”
 - status: “Active”
 - numberOfRounds: 3
 - maxPoints: 100
 - organizerId: 1
- očekivani rezultati: Sve getter metode vraćaju iste vrijednosti koje su na ulazu.
- dobiveni rezultati:



3. izvorni kod:

```
@Test
void testQuizCreation() {
    // Kreiranje Quiz-a
    Quiz quiz = new Quiz();
    quiz.setQuizName(quizName: "Kviz znanja");
    quiz.setQuizTheme(quizTheme: "Opce znanje");
    quiz.setApplicationType(applicationType: "Standard");
    quiz.setStatus(status: "Active");
    quiz.setNumberOfRounds(numberOfRounds: 3);
    quiz.setMaxPoints(maxPoints: 100);
    quiz.setOrganizerId(organizerId: 1);

    // provjera rezultata
    assertEquals("Kviz znanja", quiz.getQuizName());
    assertEquals("Opce znanje", quiz.getQuizTheme());
    assertEquals("Standard", quiz.getApplicationType());
    assertEquals("Active", quiz.getStatus());
    assertEquals(3, quiz.getNumberOfRounds());
    assertEquals(100, quiz.getMaxPoints());
    assertEquals(1, quiz.getOrganizerId());
}
```

6.1.2 redovni slučaj

1. Testiranje ispravnog mapiranja podataka iz objekta Quiz u objekt QuizResponse za ispravne ulaze vrijednosti
2. Ispitni slučaj:
 - ulazni podaci:
 - quizName: "History Quiz"
 - quizTheme: "World History"
 - maxPoints: 50
 - organizerId: 5
 - očekivani rezultati: Objekt QuizResponse sadrži iste vrijednosti koje su postavljene u objektu Quiz, bez gubitka izmjene podataka.
 - dobiveni rezultati:

Test Runner for Java

```
✓ QuizMapperTest $(symbol-namespace) com.example.demo < $(project) demo
✓ testQuizMapperWithData() $(symbol-class) QuizMapperTest < $(symbol-namespace) com.example.demo < $(project) demo
```

3. izvorni kod:

```

@Test
void testQuizMapperWithData() {
    // Test mapera koji ispravno preslikava podatke kviza u response
    Quiz quiz = new Quiz();
    quiz.setQuizName(quizName: "History Quiz");
    quiz.setQuizTheme(quizTheme: "World History");
    quiz.setMaxPoints(maxPoints: 50);
    quiz.setOrganizerId(organizerId: 5);

    QuizResponse response = QuizMapper.toResponse(quiz);

    assertEquals("History Quiz", response.getQuizName());
    assertEquals("World History", response.getQuizTheme());
    assertEquals(50, response.getMaxPoints());
    assertEquals(5, response.getOrganizerId());
}

```

6.1.3 rubni slučaj

1. Testiranje osnovne funkcionalnosti ping metode u klasi QuiController, kojom se provjerava dostupnost kontrolera
2. Ispitni slučaj:
 - ulazni podaci:
 - mock objekt SupabaseRepository
 - instanca klase ObjectMapper
 - poziv metode ping() bez ulaznih parametara
 - očekivani rezultati: Metoda ping() vraća string vrijednosti “pong”.
 - dobiveni rezultati: Metoda ping() vratila je očekivanu vrijednost “pong”.

Test Runner for Java

  QuizControllerTest \$(symbol-namespace) com.example.demo < \$(project) demo
  testPingEndpoint() \$(symbol-class) QuizControllerTest < \$(symbol-namespace) com.example.demo < \$(project) demo

3. izvorni kod:

```

@Test
void testPingEndpoint() {
    // Provjera vraca li ping endpoint "pong"
    SupabaseRepository repository = mock(SupabaseRepository.class);
    ObjectMapper objectMapper = new ObjectMapper();
    QuizController controller = new QuizController(repository, objectMapper);

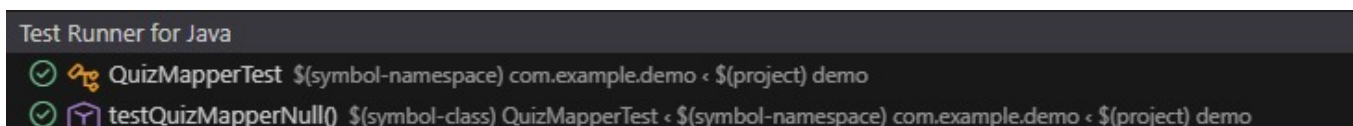
    String result = controller.ping();

    assertEquals("pong", result);
}

```

6.1.4 rubni slučaj

1. Testiranje ponašanja QuizMapper klase pri mapiranju objekta Quiz koji sadrži null vrijednosti
2. Ispitni slučaj:
 - ulazni podaci:
 - quizName: null
 - quizTheme: null
 - status: null
 - numberOfRounds: null
 - očekivani rezultati: Metoda QuizMapper.toResponse() ispravno mapira null vrijednosti iz objekta Quiz u objekt QuizResponse, bez bacanja iznimki.
 - dobiveni rezultati:



3. izvorni kod:

```
@Test
void testQuizMapperNull() {
    // Test mapera koji obrađuje null vrijednosti u Quizu
    Quiz quiz = new Quiz();
    quiz.setQuizName(quizName: null);
    quiz.setQuizTheme(quizTheme: null);
    quiz.setStatus(status: null);
    quiz.setNumberOfRounds(numberOfRounds: null);

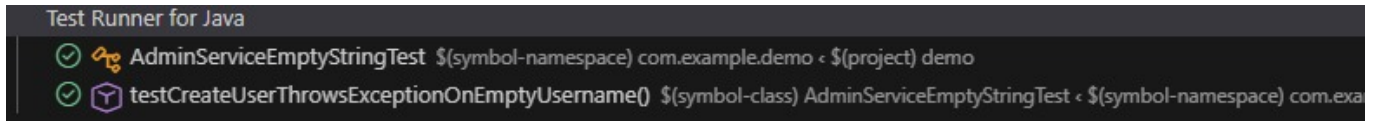
    QuizResponse response = QuizMapper.toResponse(quiz);

    assertNull(response.getQuizName());
    assertNull(response.getQuizTheme());
    assertNull(response.getStatus());
    assertNull(response.getNumberOfRounds());
}
```

6.1.5 izazivanje pogreške

1. Testiranje reakcije metode createUser() u klasi AdminService na prazno korisničko ime
2. Ispitni slučaj:
 - ulazi podaci:
 - username: ""
 - email: "test@example.com"

- očekivani rezultati: Metoda createUser() baca IllegalArgumentException zbog nevažećeg korisničkog imena.
- dobiveni rezultati: Prolaz ispitivanja. Bačena je očekivana iznimka IllegalArgumentException.



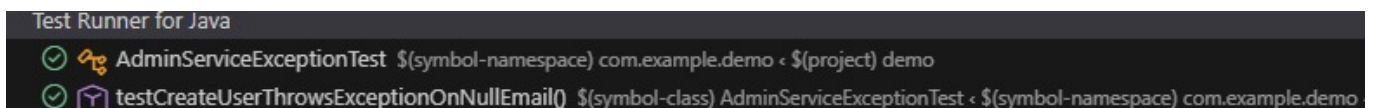
3. izvorni kod:

```
@Test
void testCreateUserThrowsExceptionOnEmptyUsername() {
    // Testiranje baca li se exception kada je korisničko ime prazno
    UserService userService = mock(UserService.class);
    SupabaseService supabaseService = mock(SupabaseService.class);
    AdminService adminService = new AdminService(userService, supabaseService);

    assertThrows(IllegalArgumentException.class, () -> {
        adminService.createUser(username: "", email: "test@example.com");
    });
}
```

6.1.6 izazivanje pogreške

1. Testiranje reakcije metode createUser() u klasi AdminService na null email adresu.
2. Ispitni slučaj:
 - ulazi podaci:
 - username: "testuser"
 - email: null
 - očekivani rezultati: Metoda createUser() baca IllegalArgumentException zbog nevažeće email adrese.
 - dobiveni rezultati: Prolaz ispitivanja. Bačena je očekivana iznimka IllegalArgumentException.



3. izvorni kod:

```

@Test
void testCreateUserThrowsExceptionOnNullEmail() {
    // Testiranje b̌aca li se exception kada je email null
    UserService userService = mock(UserService.class);
    SupabaseService supabaseService = mock(SupabaseService.class);
    AdminService adminService = new AdminService(userService, supabaseService);

    assertThrows(IllegalArgumentException.class, () -> {
        adminService.createUser(username: "testuser", email: null);
    });
}

```

6.1.7 neimplementirane funkcije

1. Provjera reakcije na poziv neimplementirane funkcionalnosti export users.
2. Ispitni slučaj
 - ulazni podaci:
 - poziv funkcionalnosti export users bez dodatnih ulaznih parametara
 - očekivani rezultati: Poziv neimplementirane funkcionalnosti rezultira bacanjem iznimke UnsupportedOperationException.
 - dobiveni rezultati: Prolaz ispitivanja. Bačena je očekivana iznimka UnsupportedOperationException.

```

✓ UnimplementedFeatureTest $(symbol-namespace) com.example.demo < $(project) demo
✓ testCallUnimplementedAdvancedSearchFeature() $(symbol-class) UnimplementedFeatureTest < $(symbol-namespace) com.example.

```

3. izvorni kod

```

@Test
void testCallUnimplementedExportUsersFeature() {
    // Poziv neimplementirane funkcionalnosti - export korisnika
    assertThrows(UnsupportedOperationException.class, () -> {
        throw new UnsupportedOperationException(message: "Export users feature is not yet implemented");
    });
}

```

6.1.8 neimplementirane funkcije

1. Provjera reakcije na poziv neimplementirane funkcionalnosti advanced search feature.
2. Ispitni slučaj
 - ulazni podaci:
 - poziv funkcionalnosti advanced search feature bez dodatnih ulaznih parametara

- očekivani rezultati: Poziv neimplementirane funkcionalnosti rezultira bacanjem iznimke `UnsupportedOperationException`.
- dobiveni rezultati: Prolaz ispitivanja. Bačena je očekivana iznimka `UnsupportedOperationException`.

```
✓ UnimplementedFeatureTest $(symbol-namespace) com.example.demo < $(project) demo
✓ testCallUnimplementedAdvancedSearchFeature() $(symbol-class) UnimplementedFeatureTest < $(symbol-namespace) com.example.
```

3. izvorni kod

```
@Test
void testCallUnimplementedAdvancedSearchFeature() {
    // Poziv neimplementirane funkcionalnosti - napredna pretraga
    assertThrows(UnsupportedOperationException.class, () -> {
        throw new UnsupportedOperationException(message: "Advanced search with filters not yet implemented");
    });
}
```

6.2 Ispitivanje sustava

6.2.1 ispitni slučaj

1. Testiranje navigacije po stranici
 2. Ispitni slučaj
- koraci:
 1. Otvoriti aplikaciju u pregledniku.
 2. Prijaviti se kao korisnik putem emaila.
 3. Isprobati rad gumbova na početnoj stranici (O aplikaciji, Admin, Kreiraj kviz, Moji kvizovi, Kvizovi, QuizWiz).
 - očekivani rezultati: Korisnik preusmjeren na odabrane stranice (O aplikaciji, Admin, Kreiraj kviz, Moji kvizovi, Kvizovi, QuizWiz).
 - dobiveni rezultati: Prikaz željenih stranica.

✓ Navigiranje po stranici		Command	Target
1	✓	open	/
2	✓	set window size	979x875
3	✓	click	css=.navbar-toggler-icon
4	✓	click	linkText=O aplikaciji
5	✓	click	linkText=Admin
6	✓	click	linkText=Kreiraj kviz
7	✓	mouse over	linkText=Moji kvizovi
8	✓	click	linkText=Moji kvizovi
9	✓	mouse out	linkText=Moji kvizovi
10	✓	click	id=featuresDropdown
11	✓	click	linkText=Kvizovi
12	✓	click	linkText=QuizWiz 🚀

6.2.2 ispitni slučaj

1. Testiranje opcije Kreiraj kviz
2. Ispitni slučaj

- koraci:

1. Otvoriti aplikaciju u pregledniku.
2. Prijaviti se kao organizator kviza.
3. Odabrati opciju Kreiraj kviz te upisati sljedeće podatke:

Naziv kviza: SeleniumTest

Tema kviza: Testiranje

Vrsta natjecanja: Tim

Datum: 13.1.2026.

Vrijeme: 10:10 AM

Opis: Samo test

Broj rundi: 1

Maksimalni bodovi: 100

Naziv lokacije: HNK

Grad: Zagreb

Kapacitet: 2

4. Odabrati gumb Kreiraj kviz.

- očekivani rezultati: Kviz kreiran s pop-up porukom obavijesti “Kviz i lokacija su uspješno kreirani!”
- dobiveni rezultati:

Kreiraj novi kviz

Podaci o kvizu

Naziv kviza*
SeleniumTest

Tema kviza*
Testiranje

Vrsta natjecanja
Tim

Datum*
01 / 13 / 2026

Vrijeme*
10:10 AM

Opis
Samo test

Broj rundi
1

Maksimalni bodovi
100

Lokacija kviza

Odaberi lokaciju (klikni na mapu)*

Map Satellite

✓ Odabrana lokacija: 45.815000, 15.981900

Adresa: Trg Josipa Langa 15, 10000, Zagreb, Hrvatska

Naziv lokacije*
HNK

Grad*
Zagreb

Kapacitet*
2

localhost:5173

Kviz i lokacija su uspješno kreirani!

OK

6.3 ispitni slučaj

1. Testiranje brisanje kviza.

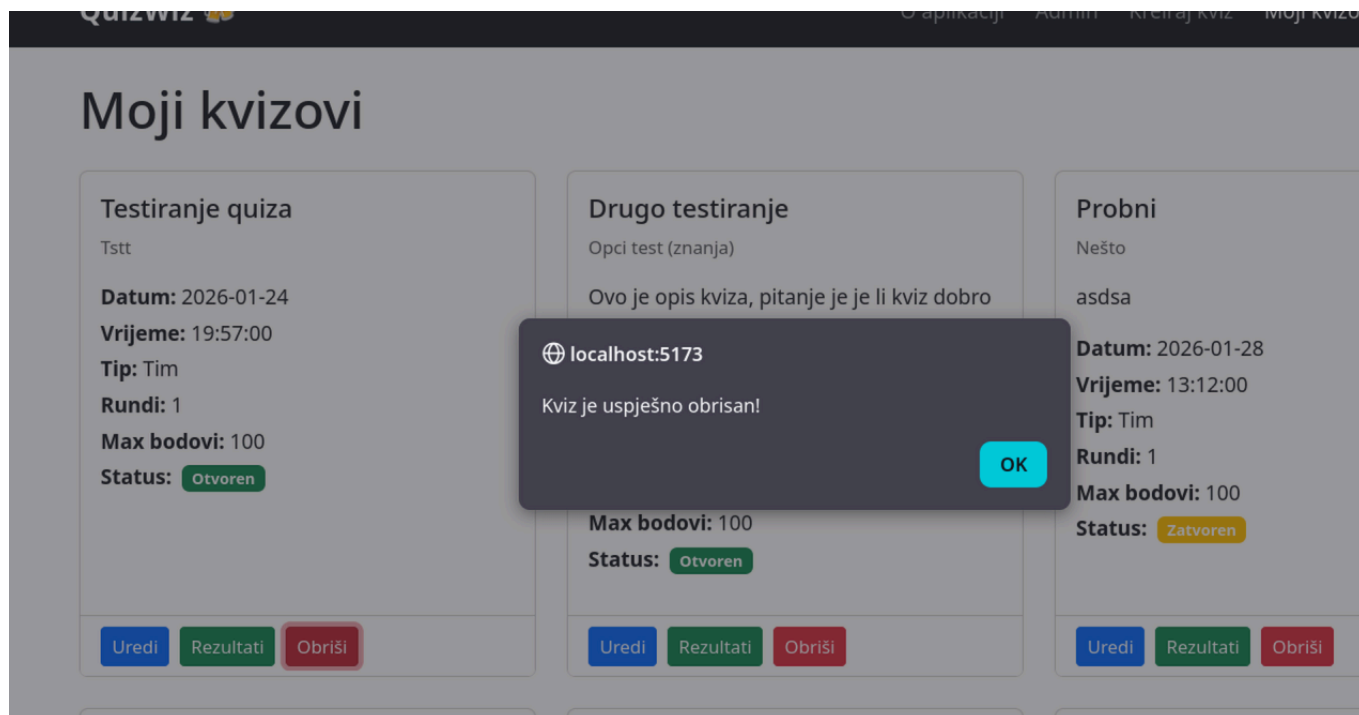
2. Ispitni slučaj

- koraci:

1. Otvoriti aplikaciju u pregledniku.
2. Prijaviti se kao organizator kviza.
3. Odabrati opciju Moji kvizovi.
4. Odabrati opciju Obriši za onaj kviz koji želite izbrisati.

- očekivani rezultati: Odabrani kviz je izbrisan te se pojavila pop-up poruka “Kviz je uspješno obrisani!”

- dobiveni rezultati:



6.4 ispitni slučaj

1. Testiranje ažuriranja timova
2. Ispitni slučaj
 - koraci:
 1. Otvoriti aplikaciju u pregledniku.
 2. Prijaviti se kao korisnik.
 3. Odabrati opciju Moji timovi.
 4. Kliknuti gumb Ažuriraj.
 - očekivani rezultati: Podaci odabranog tima se ažuriraju.
 - dobiveni rezultati:

	Command	Target	Value
1	✓ open	http://localhost:5173/my-teams	
2	✓ set window size	1703x788	
3	✓ click	css=.btn-sm:nth-child(1)	
4	✓ click	css=.mt-3 > .form-control	
5	✓ store value	css=.mt-3 > .form-control	wizionari.progi@gmail.com\nSeleniumTest@gmail.com
6	✓ click	css=.mt-3 > .form-control	
7	✓ store value	css=.mt-3 > .form-control	wizionari.progi@gmail.com\nSeleniumTest@gmail.com\nProba@gmail.com
8	✓ click	css=.btn-primary	
9	✓ click	css=.btn-sm:nth-child(1)	
10	✓ mouse down at	css=.mt-3 > .form-control	164.66668701171875,64.5333251953125
11	✓ mouse move at	css=.mt-3 > .form-control	164.66668701171875,64.5333251953125
12	✓ mouse up at	css=.mt-3 > .form-control	164.66668701171875,64.5333251953125
13	✓ click	css=.mt-3 > .form-control	
14	✓ click	css=.mt-3 > .form-control	
15	✓ store value	css=.mt-3 > .form-control	wizionari.progi@gmail.com\nSeleniumTest@gmail.com\nNovi@gmail.com
16	✓ click	css=.btn-primary	

Moji timovi

Prikaz svih timova u kojima sudjeluješ.

Naziv tima

ID: 19

wizionariproigmailcom-6-1769066325512

Prijavljen na kviz

[Quiz #6 →](#)

Bodovi Rang Broj članova

0 — 3

Članovi

- wizionari.progi@gmail.com
- SeleniumTest@gmail.com
- Novi@gmail.com

Ažuriraj

Izbriši

6.3 Rezultati ispitivanja

Provedenim ispitivanjima nisu utvrđene greške pri izvođenju definiranih akcija.

7. Korištene tehnologije i alati

1. **Programski jezici:** Projekt je razvijen korištenjem JavaScripta (ECMAScript2025) za izradu korisničkog sučelja, Jave (verzija 17) za razvoj poslužiteljskog dijela aplikacije te SQL za rad s relacijskom bazom podataka. JavaScript je odabran zbog svoje fleksibilnosti i široke podrške među modernim web preglednicima, što omogućuje izradu dinamičnih i interaktivnih korisničkih sučelja. Java pruža stabilno, sigurno i skalabilno okruženje za implementaciju poslovne logike aplikacije. SQL omogućuje učinkovito spremanje, dohvat, ažuriranje i analizu podataka, što je ključno za pouzdano i konzistentno upravljanje informacijama u aplikaciji.

2. Radni okviri i biblioteke:

- React (verzija 19): Za razvoj klijentskog dijela aplikacije korišten je React, popularna JavaScript biblioteka za izgradnju interaktivnih i responzivnih korisničkih sučelja. React omogućuje modularan razvoj putem samostalnih komponenti koje su jednostavne za ponovno korištenje i ažuriranje, čime se olakšava održavanje i skalabilnost aplikacije.
- Node.js: Node.js je runtime okruženje koje omogućuje izvršavanje JavaScript koda izvan preglednika.
- Spring Boot (verzija 4): Za razvoj backend aplikacije korišten je Spring Boot, moderan Java framework koji omogućuje brzo postavljanje i konfiguraciju servisa te integraciju s bazama podataka i sigurnosnim mehanizmima.
- OAuth 2.0: Za autentifikaciju korisnika korišten je OAuth 2.0 protokol, industrijski standard koji omogućuje sigurnu autorizaciju i odobravanje pristupa resursima putem vanjskih poslužitelja kao što je Google.

3. **Baza podataka:** Projekt koristi PostgreSQL (verzija 18) kao relacijsku bazu podataka. PostgreSQL je odabran zbog svoje robusnosti, visoke pouzdanosti i snažnog modela za rad sa strukturiranim podacima, koji omogućuje učinkovitu pohranu i upravljanje velikim količinama podataka.

4. Razvojni alati:

- Visual Studio Code (VSC) - za push/pull i commite na GitHub te pisanje dokumentacije
- Git (verzija 2)
- GitHub

5. **Alati za ispitivanje:** Pri ispitivanju aplikacije korišteni su alati: JUnit i Selenium.

6. **Alati za razmještaj:** Pri izradi projekta korišten je Docker (verzija 24) za kontejnerizaciju aplikacije što omogućuje jednostavno pokretanje aplikacije na različitim sustavima.

7. **Cloud platforma:** Za hostanje aplikacije korištene su platforme Vercel za deployment frontend aplikacije te Railway za backend. Vercel je odabran zbog automatskog povezivanja s GitHub repozitorijem, što omogućuje kontinuiranu integraciju i automatski deployment pri svakoj promjeni koda, čime se značajno ubrzava proces isporuke i testiranja frontend dijela aplikacije. Railway je korišten za hostanje backend aplikacije zbog jednostavne konfiguracije, podrške za Spring Boot aplikacije te mogućnosti jednostavnog postavljanja varijabli okoline (environment variables). Platforma omogućuje brzo pokretanje serverskih aplikacija, automatski build i deploy iz GitHub repozitorija, kao i jednostavno praćenje logova i stanja aplikacije, što olakšava razvoj i održavanje backenda. Za hostanje baze podataka korišten je Supabase, koji pruža upravljanu PostgreSQL bazu podataka kao temeljnu komponentu sustava. Osim baze podataka, Supabase nudi ugrađenu autentikaciju korisnika, upravljanje pravima pristupa (Row Level Security) te automatski generiran REST API i realtime funkcionalnosti temeljene na promjenama u bazi podataka. Zahvaljujući tim značajkama omogućena je brza, sigurna i skalabilna implementacija pohrane podataka i upravljanja korisnicima, bez potrebe za ručnom konfiguracijom i održavanjem vlastite baze podataka i infrastrukture.

8.1. Instalacija

8.1.1 Preduvjeti

Instalirane aplikacije: * Git * Node.js * JDK (Java - v17) * Docker * Maven * Spring Boot (razvojno okruženje) * React (razvojno okruženje) ## 8.1.2 Preuzimanje Prvo se klonira git stranica te uđe u datoteku stranice, zatim se naredbom npm install preuzimaju sve potrebne biblioteke iz package.json i spremaju u node_modules datoteku.

```
git clone https://github.com/MatijaBujanic/QuizWiz-TG09.1.git
cd QuizWiz-TG09.1
npm install
```

8.2 Postavke

Konfiguracijske datoteke nije potrebno prilagođavati. Također, bazu podataka nije potrebno inicijalizirati jer je baza podataka naše aplikacije Cloud hostana na Supabase.

8.3 Pokretanje aplikacije

8.3.1 Razvojno okruženje

Pokretanje frontenda

```
cd fronted
npm run dev
```

Pokretanje backenda

Potrebno je izaći iz svih poddirektorija, odnosno nalaziti se u glavnom QUIzWiz-TG09.1 direktoriju.

```
cd backend/demo
mvn spring-boot:run
```

8.3.2 Produkcijsko okruženje

Prevođenje aplikacije:

```
npm run build
```

Pokretanje poslužitelja:

```
mvn spring-boot:run
```

Provjera rada:

Lokalno se aplikacija može testirati na <http://localhost:8080>. Nakon pokretanja poslužitelja moguće je također dokumentirati i testirati API-je na stranici <http://localhost:8080/swagger-ui/index.html>.

8.4 Upute za administratore

8.4.1 Pristup administratorskom sučelju

Administratorskom sučelju se pristupa preko stranice za prijavu gdje se odabere prijava putem Google poslužitelja te se upišu sljedeći podaci:

```
username: wizionari.progi@gmail.com
```

```
password: wizionariadmin2025!
```

8.4.2 Redovito održavanje

Sam pregled logova nije dostupan. Prije ažuriranja aplikacije potrebno se nalaziti u kloniranom repozitoriju. Aplikaciju je preporučeno ažurirati barem jednom mjesečno pozivanjem naredbi:

```
git pull origin main
```

```
npm install
```

```
cd frontend
```

```
npm install
```

```
npm run build
```

```
cd ../backend/demo
```

```
mvn clean install
```

```
mvn spring-boot:run
```

8.5 Opis pristupa aplikaciji na javnom poslužitelju

Aplikaciji se pristupa putem: quiz-wiz-tg-09-1.vercel.app. Funkcionalnosti i prijava admina su iste kao i kod lokalne verzije.

9.1 Tehnički izazovi

Tijekom izrade projekta susreli smo se s brojnim tehničkim izazovima, ponajprije zbog nedovoljnog poznavanja alata i tehnologija koje smo koristili. Pojedine funkcionalnosti zahtijevale su više vremena za razumijevanje i implementaciju nego što smo u početku očekivali. Osim toga, tehničke poteškoće često su dodatno otežavale napredak, što je ponekad dovodilo do zastoja u radu i potrebe za dodatnim istraživanjem i testiranjem rješenja.

9.2 Stečena znanja i iskustva

Rad na ovom projektu omogućio nam je stjecanje vrijednih znanja i iskustva, kako u tehničkom, tako i u organizacijskom smislu. Naučili smo koristiti nove alate, bolje razumijeti procese razvoja te prepoznati važnost timskog rada. Posebno smo uvidjeli koliko su komunikacija, suradnja i međusobna podrška ključni za uspješnu realizaciju zajedničkog cilja. Ovo iskustvo pomoglo nam je da osvijestimo vlastite prednosti, ali i područja na kojima se trebamo dodatno razvijati.

9.3 Lakše izvođenje projekta

Smatramo da bi se projekt mogao provesti znatno učinkovitije uz bolju organizaciju rada, jasniju podjelu zadataka i pravovremenu komunikaciju među članovima tima. Definiranje uloga od samog početka, redoviti sastanci i otvorena razmjena informacija doprinijeli bi bržem rješavanju problema i smanjenju nesporazuma. Također, prethodno upoznavanje s korištenim alatima omogućilo bi sigurniji i učinkovitiji pristup samoj realizaciji projekta.

9.4 Perspektive za nastavak rada

Trenutno, razvojni tim Wizionari nema namjeru nastaviti s razvojem web aplikacije nakon završetka svih aktivnosti FER kolegija “Programsko inženjerstvo”.

9.5 Popis neimplementiranih funkcionalnosti

	ID zahtjeva	Opis
F-001.1		Korisnici se mogu registrirati putem obrasca s osnovnim podacima (korisničko ime, lozinka, e-mail, naziv tima, kontakt podaci) i dobiti potvrdu o uspješnoj registraciji. Korisnik također može odabrati opciju registracije tima, dodaje naziv i članove tima.
F-008.1		Organizatori mogu odgovarati na recenziju i davati povratne informacije.

A. Dnevnik promjene dokumentacije

Rev.	Opis promjene/dodatka	Autori	Datum
0.1	Napravljen predložak	Matija Bujanić	23.10.2025.
0.2	Popunjen opis projektnog zadatka	Mihael Vranić	26.10.2025.
0.3	Popunjena analiza zahtjeva	Paula Vidak	26.10.2025.
0.4	Popunjeni obrasci uporabe	Dario Herceg	27.10.2025.
0.5	Popunjeni dijagrami obrazaca uporabe	Dario Herceg	28.10.2025.
0.6	Napravljeni sekvencijski dijagrami	Jona Matsumoto Šegota	29.10.2025.
0.7	Napravljena arhitektura i dizajn sustava do dijagrama razreda	Paula Vidak	4.11.2025.
0.8	Napravljen dijagram razreda	Dario Herceg	10.11.2025.
0.9	Dorađen popis sastanaka i tablica aktivnosti grupe	Paula Vidak	12.11.2025.
1.1	Prepravljene greške iz prve predaje	Paula Vidak	13.1.2026.
1.2	Napisane korištene tehnologije za implementaciju aplikacije	Paula Vidak	13.1.2026.
1.3	Napravljeni dijagrami stanja	Paula Vidak	13.1.2026.
1.4	Napravljeni dijagrami komponenata i razmještaja	Paula Vidak	14.1.2026.
1.5	Napravljeni dijagrami aktivnosti	Dario Herceg	14.1.2026.
1.6	Napisan zaključak i budući rad	Paula Vidak	15.1.2026.
1.7	Napisano ispitivanje programskog rješenja	Paula Vidak	23.1.2026.
1.8	Napisane upute za puštanje u pogon	Paula Vidak	23.1.2026.

B. Dnevnik sastajanja

1. sastanak

- Datum: 9. listopada 2025.
- Prisustvovali: M.Bujanić, A.Kasanić, J.M.Šegota, M.Vranić, D.Herceg, F.Gojak, P.Vidak
- Teme sastanka:
 - uspostavljena WhatsApp grupa svih članova grupe
 - uspostavljena Discord grupa sa svim članovima
 - dogovorena tema zadatka projekta

2. sastanak

- Datum: 21. listopada 2025.
- Prisustvovali: M.Bujanić, A.Kasanić, J.M.Šegota, M.Vranić, D.Herceg, P.Vidak
- Teme sastanka:
 - dogovorena raspodjela zadataka unutar tima
 - dogovoreno ime tima i aplikacije
 - započeta izrada Github repozitorija

3. sastanak

- Datum: 23. listopada 2025.
- Prisustvovali: M.Bujanić, M.Vranić, D.Herceg, P.Vidak
- Teme sastanka:
 - pregled trenutnog napretka u razvoju
 - započet sprint
 - detaljno razrađeni opis projektnog zadatka ([issue #2](#)), analiza zahtjeva ([issue #3](#)) i specifikacija zahtjeva sustava ([issue #4](#))

4. sastanak

- Datum: 30. listopada 2025.
- Prisustvovali: M.Bujanić, M.Vranić, D.Herceg, J.M.Šegota, F.Gojak, A.Kasanić, P.Vidak
- Teme sastanka:
 - detaljnije razrađivanje arhitekture i dizajna sustava na Github wiki stranici ([issue #6](#))
 - proučavanje alata za frontend
 - dogovor za razvoj dijagrama baze podataka
 - početak rada kostura za backend ([issue #5](#))

5. sastanak

- Datum: 3. studenog 2025.
- Prisustvovali: A.Kasanić, M.Vranić
- Teme sastanka:
 - finalna realizacija i konfiguracija modela baze podataka ([issue #11](#))

6. sastanak

- Datum: 6. studenog 2025.
- Prisustvovali: M.Bujanić, M.Vranić, D.Herceg, A.Kasanić, J.M.Šegota, F.Gojak, P.Vidak
- Teme sastanka:
 - dogovor oko deploymenta
 - dogovor oko finalnog izgleda frontenda ([issue #9](#) i [issue #7](#))
 - dogovor za deployment baze preko Supabase ([issue #10](#))
 - dodana vizualna ploča “Plan rada” na Github repozitorij
 - popravljjanje i uređivanje dokumentacije na Github repozitoriju ([issue #8](#))

7. sastanak

- Datum: 13. studenog 2025.
- Prisustvovali: M.Bujanić, A.Kasanić, F.Gojak, J.M.Šegota, D.Herceg, P.Vidak
- Teme sastanka:
 - pregled trenutnog napretka u razvoju
 - dogovor za dodatnu funkcionalnost - admin stranica i admin login ([issue #13](#))

8. sastanak

- Datum: 11. prosinca 2025.
- Prisustvovali: D.Herceg, P.Vidak, M. Vranić
- Teme sastanka:
 - pregled prve predaje projekta

9. sastanak

- Datum: 19. prosinca 2025.
- Prisustvovali: M.Bujanić, A.Kasanić, J.M.Šegota, M.Vranić, D.Herceg, F.Gojak, P.Vidak
- Teme sastanka:
 - dogovor za daljnji rad - potrebne funkcionalnosti, izgled aplikacije...

10. sastanak

- Datum: 10. siječnja 2026.

- Prisustvovali: M.Bujanić, A.Kasanić, J.M.Šegota, M.Vranić, D.Herceg, F.Gojak, P.Vidak
- Teme sastanka:
 - daljnja raspodjela zadataka unutar tima ([issue #22](#) i [issue #23](#))

11. sastanak

- Datum: 13. siječnja 2026.
- Prisustvovali: A. Kasanić, M. Vranić
- Teme sastanka:
 - dogovor za implementaciju baze i početak rada na endpointovima

12. sastanak

- Datum: 15. siječnja 2026.
- Prisustvovali: M.Bujanić, A.Kasanić, M.Vranić, D.Herceg, F.Gojak, P.Vidak
- Teme sastanka:
 - pregled dosadašnjeg napretka
 - usklađivanje dokumentacije i rada aplikacije ([issue #28](#))
 - dogovor oko povezivanja frontenda i backenda ([issue #24](#))
 - dogovor povezivanja funkcionalnosti i izgleda određenih stranica ([issue #25](#), [issue #26](#) i [issue #27](#))
 - dogovor za ispitivanje aplikacije

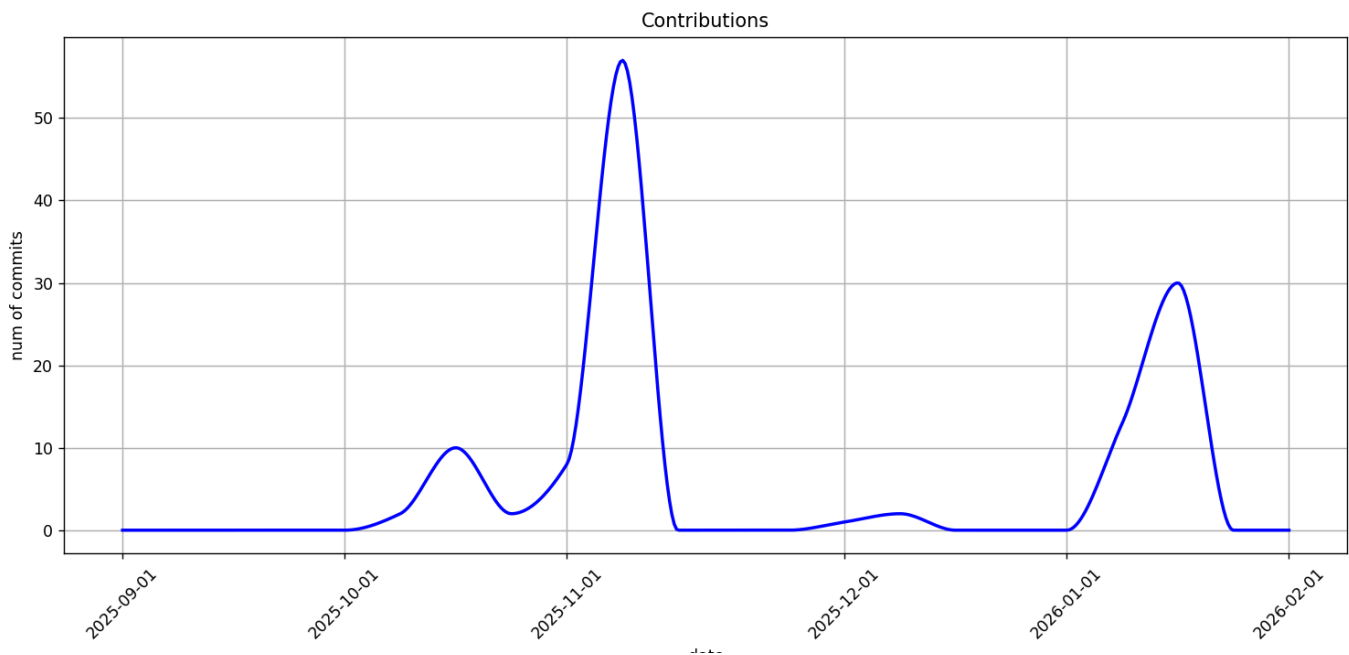
Plan rada

Tablica aktivnosti

aktivnost	M.Bujanić	A.Kasanić	M.Vranić	J.M.Šegota	F.Gojak	D.Herceg	P.Vidak
Upravljanje projektom	15						
Opis projektnog zadatka		3	4				
Funkcionalni zahtjevi			1				4
Opis pojedinih obrazaca						9	
Dijagram obrazaca						6	
Sekvencijski dijagrami				8			
Opis ostalih zahtjeva							5
Arhitektura i dizajn sustava						1	7
Baza podataka		3	3				

aktivnost	M.Bujanić	A.Kasanić	M.Vranić	J.M.Šegota	F.Gojak	D.Herceg	P.Vidak
Dijagram razreda						8	
Dijagram stanja							3
Dijagram aktivnosti						3	
Dijagram komponenti							2
Korištene tehnologije i alati		3	2				2
Ispitivanje programskog rješenja						6	
Dijagram razmještaja							2
Upute za puštanje u pogon							3
Dnevnik sastajanja							2
Zaključak i budući rad							2
Popis literature							1
Frontend				25	22	3	
Backend	15	25	22				
Izrada početne stranice					3		
Izrada baze podataka		2	2				
Spajanje s bazom podataka	3	3	2		7		
Spajanje frontenda i backenda	5			5	7		
Deploy aplikacije	10						
Izrada prezentacije							3
Dokumentacija (README, CONTRIBUTING...)			2			2	3
Ukupno	48	39	38	38	39	38	39

Dijagram pregleda promjena



Ključni izazovi i rješenja

Naš tim je tijekom rada na ovom projektu ponajviše uvidio koliko je zapravo zahtjevno učinkovito funkcionirati u grupi. Iako smo u početku imali motivaciju i želju za uspjehom, ubrzo smo shvatili da su organizacija rada i međusobna komunikacija ključni faktor koji mogu znatno utjecati na konačni rezultat. Upravo su ti elementi predstavljali jedne od najvećih izazova tijekom izrade projekta, uz dodatni problem nedovoljnog poznavanja alata i tehnologije koje smo koristili. Kad bi se neki zadatak pokazao prezahtjevnim za samostalan rad pojedinca, nastojali smo ga rješavati zajedničkim snagama. Takav pristup ponekad je donosio dobre rezultate, dok u nekim situacijama nije bio jednako uspješan, no unatoč tome trudili smo se održavati timski duh i međusobnu podršku. Kroz taj proces naučili smo koliko je važno jasno definirati uloge, odgovornosti i očekivanja među članovima tima. Smatramo da je projekt mogao biti izveden uspješnije, ali ne zbog nedostatka truda ili zalaganja, već ponajprije zbog manjka iskustva u timskom radu. Nedovoljno razvijene vještine u kvalitetnoj raspodjeli zadataka, planiranje vremena i pravovremenoj komunikaciji o problemima često su nas usporavale. Ipak, ovo iskustvo smatramo vrijednom lekcijom koja će nam pomoći u budućim projektima, jer smo stekli bolji uvid u važnost suradnje, odgovornosti i otvorene komunikacije unutar tima.

C. Literatura

1. Programsko inženjerstvo, FER ZEMRIS, <http://www.fer.hr/predmet/proinz>
2. I. Sommerville, "Software engineering", 8th ed, Addison Wesley, 2007.
3. T.C.Lethbridge, R.Langaniere, "Object-Oriented Software Engineering", 2nd ed. McGraw-Hill, 2005.
4. I. Marsic, Software engineering book“, Department of Electrical and Computer Engineering, Rutgers University, <http://www.ece.rutgers.edu/~marsic/books/SE>
5. The Unified Modeling Language, <https://www.uml-diagrams.org/>
6. Astah Community, <http://astah.net/editions/uml-new>
7. Architect modern web applications with ASP.NET core and azure - .NET, Architect modern web applications with ASP.NET Core and Azure - .NET | Microsoft Learn. Dostupno: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/> (7. studeni 2024.).
8. Fowler, M. & Lewis, J., Microservices – a definition of this new architectural term., 2014.
9. Newman, S., Building Microservices, 2015., O'Reilly Media